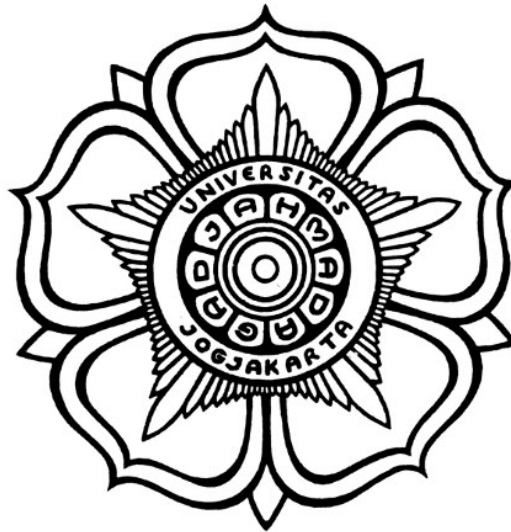


**PERANCANGAN PERIPHERAL I2S DENGAN AXI4-LITE
INTERFACE MENGGUNAKAN VERILOG**

SKRIPSI



THE SUSTAINABLE DEVELOPMENT GOALS
Industry, Innovation and Infrastructure
Affordable and Clean Energy
Climate Action

Disusun oleh:

Muhammad Muqtada Alhaddad
22/500341/TK/54841

**PROGRAM SARJANA PROGRAM STUDI TEKNOLOGI INFORMASI
DEPARTEMEN TEKNIK ELEKTRO DAN TEKNOLOGI INFORMASI
FAKULTAS TEKNIK UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026**

HALAMAN PENGESAHAN

PERANCANGAN PERIPHERAL I2S DENGAN AXI4-LITE INTERFACE MENGGUNAKAN VERILOG

SKRIPSI

Diajukan Sebagai Salah Satu Syarat untuk Memperoleh
Gelar Sarjana Teknik
pada Departemen Teknik Elektro dan Teknologi Informasi
Fakultas Teknik
Universitas Gadjah Mada

Disusun oleh:

Muhammad Muqtada Alhaddad
22/500341/TK/54841

Telah disetujui dan disahkan

Pada tanggal

Dosen Pembimbing I

Dosen Pembimbing II

Ir. Agus Bejo, S.T., M.Eng., D.Eng., IPM.

Ir. Wahyu Dewanto, M.T.

PERNYATAAN BEBAS PLAGIASI

Saya yang bertanda tangan di bawah ini :

Nama : Muhammad Muqtada Alhaddad
NIM : 22/500341/TK/54841
Tahun terdaftar : 2022
Program : Sarjana
Program Studi : Teknologi Informasi
Fakultas : Teknik Universitas Gadjah Mada

Menyatakan bahwa dalam dokumen ilmiah Skripsi ini tidak terdapat bagian dari karya ilmiah lain yang telah diajukan untuk memperoleh gelar akademik di suatu lembaga Pendidikan Tinggi, dan juga tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang/lembaga lain, kecuali yang secara tertulis disitasi dalam dokumen ini dan disebutkan sumbernya secara lengkap dalam daftar pustaka.

Dengan demikian saya menyatakan bahwa dokumen ilmiah ini bebas dari unsur-unsur plagiasi dan apabila dokumen ilmiah Skripsi ini di kemudian hari terbukti merupakan plagiasi dari hasil karya penulis lain dan/atau dengan sengaja mengajukan karya atau pendapat yang merupakan hasil karya penulis lain, maka penulis bersedia menerima sanksi akademik dan/atau sanksi hukum yang berlaku.

Yogyakarta, 26 Mei 2026

Materai Rp10.000

(Tanda tangan)

Muhammad Muqtada Alhaddad
22/500341/TK/54841

HALAMAN PERSEMBAHAN

Skripsi ini penulis persembahkan kepada:

Orang tua tercinta atas doa, dukungan, dan pengorbanan tanpa henti.

Para dosen pembimbing yang telah membimbing perjalanan akademik penulis.

Semua pihak yang telah berkontribusi dalam kesuksesan penelitian ini.

KATA PENGANTAR

"Dan tidaklah kamu diberi ilmu melainkan hanya sedikit." (QS. Al-Isra: 85)

"And of knowledge, you have been given only a little." (Quran 17:85)

Puji syukur ke hadirat Allah SWT atas limpahan rahmat, karunia, serta petunjuk-Nya sehingga tugas akhir berupa penyusunan skripsi ini dengan judul **"Perancangan Peripheral I2S dengan AXI4-Lite Interface menggunakan Verilog"** telah terselesaikan dengan baik. Dalam hal penyusunan tugas akhir ini penulis telah banyak mendapatkan arahan, bantuan, serta dukungan dari berbagai pihak. Oleh karena itu pada kesempatan ini penulis mengucapkan terima kasih kepada:

1. Kedua orang tua dan keluarga besar yang senantiasa memberikan doa, dukungan moril, serta kasih sayang yang tak terhingga sepanjang masa studi penulis.
2. Bapak Ir. Agus Bejo, S.T., M.Eng., D.Eng., IPM. dan Bapak Ir. Wahyu Dewanto, M.T. selaku dosen pembimbing yang telah memberikan bimbingan, arahan, serta waktu luangnya untuk membantu penulis menyelesaikan penelitian ini.
3. Segenap dosen dan staf di Departemen Teknik Elektro dan Teknologi Informasi (DTETI) FT UGM yang telah memberikan bekal ilmu pengetahuan dan bantuan administratif selama masa perkuliahan.
4. Teman-teman seperjuangan Teknologi Informasi angkatan 2022 yang telah memberikan semangat dan kerja sama selama masa studi.

Akhir kata penulis berharap semoga skripsi ini dapat memberikan manfaat bagi perkembangan ilmu pengetahuan, khususnya di bidang sistem digital dan sistem embedded, serta bagi kita semua, aamiin.

Yogyakarta, 14 Mei 2026

Muhammad Muqtada Alhaddad

DAFTAR ISI

HALAMAN PENGESAHAN	ii
PERNYATAAN BEBAS PLAGIASI	iii
HALAMAN PERSEMBAHAN	iv
KATA PENGANTAR	v
DAFTAR ISI	vi
DAFTAR TABEL	x
DAFTAR GAMBAR	xi
DAFTAR SINGKATAN.....	xii
INTISARI.....	xiii
ABSTRACT	xiv
BAB I Pendahuluan	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Tujuan Penelitian	3
1.3.1 Tujuan perancangan dan integrasi perangkat keras	3
1.3.2 Tujuan integrasi SoC dan perangkat lunak.....	3
1.3.3 Tujuan verifikasi dan validasi.....	3
1.4 Batasan Penelitian	3
1.4.1 Batasan perangkat keras dan RTL	4
1.4.2 Batasan SoC dan perangkat lunak.....	4
1.4.3 Batasan pengujian dan dokumentasi	4
1.5 Manfaat Penelitian	4
1.5.1 Manfaat akademik	4
1.5.2 Manfaat praktis	5
1.5.3 Manfaat pengembangan keahlian	5
1.6 Sistematika Penulisan.....	5
BAB II Tinjauan Pustaka dan Dasar Teori	6
2.1 Tinjauan Pustaka	6
2.1.1 Penelitian tentang implementasi I2S pada FPGA	6
2.1.2 Penelitian tentang prosesor soft-core dan integrasi SoC	6
2.1.3 Penelitian tentang sinkronisasi antar-domain clock (CDC)	7
2.1.4 Gap penelitian dan kontribusi	7
2.2 Dasar Teori	8
2.2.1 Arsitektur RISC-V dan MicroBlaze V	8
2.2.1.1 Model Memory-Mapped I/O (MMIO).....	8
2.2.1.2 Register Kontrol dan Status (CSR)	8

2.2.2	Protokol Komunikasi AXI4-Lite	9
2.2.2.1	Kanal dan Sinyal AXI4-Lite	9
2.2.2.2	Mekanisme Handshake VALID/READY	9
2.2.3	Protokol komunikasi I2S	9
2.2.3.1	Sinyal I2S	9
2.2.3.2	Format data Philips I2S.....	10
2.2.3.3	Relasi <i>sample rate</i> , BCLK, dan format slot	10
2.2.4	DAC audio PCM5102A.....	11
2.2.5	FPGA dan Basys3.....	11
2.2.6	MicroBlaze V dan memory-mapped I/O.....	11
2.2.7	Antarmuka AXI4-Lite	12
2.2.8	Sinkronisasi lintas domain clock (CDC)	12
2.2.9	Clocking Wizard dan frekuensi aktual	13
2.2.10	Verilog HDL dan perancangan RTL	13
2.2.11	Perbandingan dengan IP I2S bawaan AMD/Xilinx	13
2.2.12	Analisis pemilihan metode	14
BAB III Metode Penelitian.....		15
3.1	Alat dan Bahan.....	15
3.1.1	Perangkat keras.....	15
3.1.1.1	Board FPGA	15
3.1.1.2	Modul DAC audio	15
3.1.1.3	Perangkat bantu	15
3.1.2	Perangkat lunak	15
3.2	Metode yang Digunakan.....	15
3.2.1	Metode perancangan hardware	15
3.2.2	Metode verifikasi dan simulasi	16
3.2.3	Metode integrasi SoC	17
3.2.4	Metode pengembangan software	17
3.3	Alur Tugas Akhir	17
BAB IV Perancangan dan Implementasi		19
4.1	Spesifikasi dan arsitektur IP I2S TX	19
4.1.1	Daftar fitur IP.....	19
4.1.2	Blok diagram perancangan IP	19
4.1.3	Register map	20
4.2	Implementasi RTL dan Integrasi Vivado.....	20
4.2.1	Pemetaan pin (constraint).....	20
4.2.2	Detail logika RTL (serializer dan CDC)	21
4.2.2.1	Mekanisme Clock Domain Crossing (CDC)	21
4.2.2.2	Logika serializer Philips I2S	21

4.3	Konfigurasi Clocking Wizard	22
BAB V	Hasil dan Pembahasan.....	23
5.1	Pengujian Sinyal	23
5.2	Analisis Frekuensi Sampling	23
5.3	Identifikasi dan Perbaikan Bug Firmware	23
5.3.1	Bug 1: Masker Merusak Bit Tanda Sampel.....	23
5.3.2	Bug 2: Timing Loop Berbasis usleep()	24
5.4	Verifikasi Setelah Perbaikan	24
5.5	Pembahasan terhadap Tujuan Penelitian	24
5.6	Perbandingan dengan referensi desain	24
BAB VI	Kesimpulan dan Saran.....	25
6.1	Kesimpulan.....	25
6.2	Saran.....	25
	DAFTAR PUSTAKA.....	27
	LAMPIRAN	L-1
L.1	Hasil resource utilization	L-1
L.1.1	Post-implementation resource report.....	L-1
L.1.2	Timing report.....	L-1
L.2	Timing diagram	L-1
L.2.1	I2S audio transmission timing	L-1
L.3	Log hasil pengujian	L-2
L.3.1	I2S audio playback test	L-2
L.4	Deskripsi block design.....	L-2
L.4.1	Diagram Block (Color-coded).....	L-3
L.4.2	I2S IP Block (Standalone)	L-4
L.4.3	Clocking View.....	L-4
L.4.4	Interface View	L-4
L.5	Hasil pengujian i2s transmitter - capture osiloskop	L-5
L.5.1	Test Frekuensi Audio	L-5
L.5.2	Test Kontrol Enable	L-6
L.5.3	Test Kontrol Mute	L-7
L.5.4	Test Routing Stereo	L-8
L.5.5	Test Lebar Sample (Sample Width)	L-9
L.6	Design Timing Summary	L-10
L.7	Power Summary.....	L-10
	Block Design IP Summaries	L-12
BAB L	Kode Sumber Perangkat Lunak (Vitis)	L-14
BAB L	Kode Sumber Perangkat Keras (Verilog)	L-18
L.1	Top-level Wrapper (i2s.v)	L-18

DAFTAR TABEL

Tabel 4.1	Register map IP I2S AXI4-Lite.	20
Tabel 4.2	Bitfield register <code>CONTROL</code>	20
Tabel 4.3	Konfigurasi clock hasil Clocking Wizard.....	22
Tabel 5.1	Hasil pengujian clock I2S pada keluarga frekuensi 48 kHz.	23
Tabel 5.2	Hasil verifikasi firmware setelah perbaikan bug.....	24
Tabel L.1	Ringkasan utilisasi resource FPGA.....	L-1
Tabel L.2	Hasil analisis timing.....	L-2

DAFTAR GAMBAR

Gambar 1.1	Ringkasan arsitektur sistem SoC dengan IP I2S TX AXI4-Lite dan DAC PCM5102A.	2
Gambar 2.2	Diagram sinyal I2S Philips Standard (Sumber: NXP)	10
Gambar 3.3	Diagram alir tahapan penelitian tugas akhir.....	18
Gambar 4.4	Blok diagram internal IP I2S TX.	19
Gambar 4.5	Block Design Vivado: MicroBlaze V terhubung ke IP I2S melalui AXI SmartConnect.	21
Gambar 5.6	Tangkapan waveform: BCLK, WS, dan DATA – satu frame stereo 64 BCLK.	23
Gambar 5.7	Keluaran audio setelah perbaikan: kanal L dan R identik.	24
Gambar 5.8	Ringkasan timing desain.	24
Gambar L.1	Ringkasan utilisasi resource (visual).	L-1
Gambar L.2	Diagram block warna-warni arsitektur SoC.....	L-3
Gambar L.3	Blok IP I2S standalone.....	L-4
Gambar L.4	Clocking view dan jalur pemilihan clock audio.	L-4
Gambar L.5	Interface view pemetaan pin dan jalur debug.	L-5
Gambar L.6	Capture osiloskop: audio tone 1 kHz.....	L-5
Gambar L.7	Capture osiloskop: audio tone 2 kHz.....	L-6
Gambar L.8	Capture osiloskop: audio tone 5 kHz.....	L-6
Gambar L.9	Capture osiloskop: enable=0.	L-6
Gambar L.10	Capture osiloskop: enable=1.	L-7
Gambar L.11	Capture osiloskop: mute=0.	L-7
Gambar L.12	Capture osiloskop: mute=1.	L-7
Gambar L.13	Hasil capture osiloskop: Left channel only - hanya kanal left dioutput.....	L-8
Gambar L.14	Hasil capture osiloskop: Right channel only - hanya kanal right dioutput.....	L-8
Gambar L.15	Hasil capture osiloskop: Both channels - kedua kanal left dan right aktif simultan	L-8
Gambar L.16	Hasil capture osiloskop: Sample width 16-bit - format audio PCM 16-bit	L-9
Gambar L.17	Hasil capture osiloskop: Sample width 24-bit - format audio PCM 24-bit pada slot 32-bit	L-9
Gambar L.18	Hasil capture osiloskop: Sample width 32-bit - full 32-bit slot I2S frame	L-10
Gambar L.19	Design timing summary report - Worst Negative Slack (WNS), Total Negative Slack (TNS), dan status timing closure	L-10
Gambar L.20	Power summary report - total on-chip power consumption, breakdown per resource (MMCM, Block RAM, Slice Logic), dan power supply margins	L-11

DAFTAR SINGKATAN

AXI	=	Advanced eXtensible Interface
BCLK	=	Bit Clock
BRAM	=	Block RAM
CDC	=	Clock Domain Crossing
CPU	=	Central Processing Unit
DAC	=	Digital to Analog Converter
DMA	=	Direct Memory Access
DSP	=	Digital Signal Processor / Processing
FF	=	Flip-Flop
FIFO	=	First-In-First-Out
FPGA	=	Field Programmable Gate Array
HDL	=	Hardware Description Language
I2S	=	Inter-IC Sound
ILA	=	Integrated Logic Analyzer
IP	=	Intellectual Property
IRQ	=	Interrupt Request
LRCK	=	Left/Right Clock (sama dengan WS)
LSB	=	Least Significant Bit
LUT	=	Look-Up Table
MCLK	=	Master Clock
MMCM	=	Mixed-Mode Clock Manager
MSB	=	Most Significant Bit
RISC-V	=	Reduced Instruction Set Computer V
RTL	=	Register Transfer Level
SoC	=	System on Chip
UART	=	Universal Asynchronous Receiver-Transmitter
WS	=	Word Select
XDC	=	Xilinx Design Constraints

INTISARI

Penelitian ini berfokus pada perancangan dan implementasi peripheral transmitter I2S (Inter-IC Sound) menggunakan bahasa Verilog HDL yang diintegrasikan ke dalam System-on-Chip (SoC) berbasis prosesor MicroBlaze V pada FPGA. Peripheral ini diimplementasikan sebagai custom IP dengan antarmuka AXI4-Lite slave sehingga dapat dikonfigurasi dan diisi data audio melalui register memory-mapped oleh perangkat lunak yang berjalan di Xilinx Vitis. Platform target yang digunakan adalah board FPGA Basys3, sedangkan keluaran audio direalisasikan melalui modul DAC PCM5102A. Modul pendukung seperti UART dan infrastruktur clock hanya digunakan sebagai penunjang sistem, sementara kontribusi utama skripsi ini adalah desain RTL, integrasi, dan validasi peripheral transmitter I2S.

Metodologi penelitian mencakup: (1) desain RTL Verilog untuk transmitter I2S yang mengikuti format Philips I2S untuk transmisi audio stereo, (2) perancangan antarmuka register AXI4-Lite untuk kontrol, pemuatan sampel, dan observasi status, (3) implementasi buffer berbasis FIFO dan interupsi watermark untuk mengurangi risiko underrun, (4) integrasi custom IP I2S ke dalam SoC MicroBlaze V menggunakan Vivado block design, (5) pengembangan perangkat lunak uji dalam bahasa C pada Xilinx Vitis untuk konfigurasi register, prefill FIFO, dan pengiriman sampel audio, serta (6) verifikasi melalui synthesis, implementation, pengamatan Integrated Logic Analyzer (ILA), dan pengujian hardware dengan modul PCM5102A.

Hasil penelitian menunjukkan bahwa IP transmitter I2S yang dirancang dapat diintegrasikan dengan baik ke dalam SoC berbasis MicroBlaze V dan dioperasikan melalui kontrol memory-mapped AXI4-Lite. Desain yang diimplementasikan mampu membangkitkan sinyal I2S yang diperlukan, mentransmisikan sampel audio stereo dalam format Philips I2S, serta mendukung dua mode frekuensi sampling nominal sekitar 48 kHz dan 96 kHz yang diturunkan dari konfigurasi audio clock. Validasi perangkat keras menunjukkan bahwa desain bekerja sesuai tujuan pada platform yang diuji dan dapat menjadi dasar yang dapat digunakan kembali untuk sistem keluaran audio digital berbasis FPGA.

Kata kunci : MicroBlaze V, I2S, Verilog, AXI4-Lite, FPGA, SoC, PCM5102A, Audio Digital

ABSTRACT

This research focuses on the design and implementation of an I2S (Inter-IC Sound) transmitter peripheral using Verilog HDL, integrated into a System-on-Chip (SoC) based on the MicroBlaze V processor on FPGA. The peripheral is implemented as a custom AXI4-Lite slave IP so that it can be configured and supplied with audio data through memory-mapped registers from software running in Xilinx Vitis. The target platform is the Basys3 FPGA board, while audio output is generated through a PCM5102A DAC module. Supporting modules such as UART and clocking infrastructure are included only as system support, whereas the main contribution of the thesis is the RTL design, integration, and validation of the I2S transmitter peripheral.

The methodology includes: (1) Verilog RTL design of an I2S transmitter that follows the Philips I2S format for stereo audio transmission, (2) design of an AXI4-Lite register interface for control, sample loading, and status observation, (3) implementation of FIFO-based buffering and watermark interrupt support to reduce underrun risk, (4) integration of the custom I2S IP into a MicroBlaze V SoC using Vivado block design, (5) development of C test software in Xilinx Vitis for register configuration, FIFO prefill, and audio sample transmission, and (6) verification through synthesis, implementation, Integrated Logic Analyzer (ILA) observation, and hardware testing with the PCM5102A module.

The results show that the designed I2S transmitter IP can be integrated successfully into a MicroBlaze V-based SoC and operated through AXI4-Lite memory-mapped control. The implemented design is able to generate the required I2S signals, transmit stereo audio samples in Philips I2S format, and support two nominal sampling-rate modes around 48 kHz and 96 kHz derived from the audio clock configuration. Hardware validation indicates that the design functions as intended on the tested platform and provides a reusable basis for FPGA-based digital audio output systems.

Keywords: MicroBlaze V, I2S, Verilog, AXI4-Lite, FPGA, SoC, PCM5102A, Digital Audio

BAB I

PENDAHULUAN

1.1 Latar Belakang

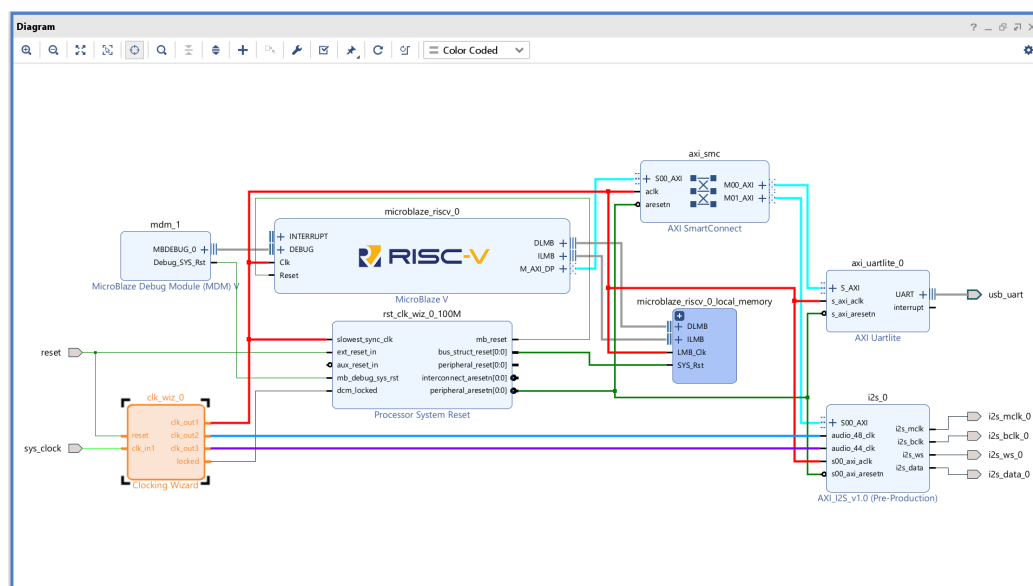
Perkembangan sistem embedded modern mendorong kebutuhan akan antarmuka audio digital yang fleksibel, dapat dikonfigurasi melalui perangkat lunak, dan mudah diintegrasikan ke dalam arsitektur System-on-Chip (SoC). Pada berbagai aplikasi seperti audio playback, perangkat edukasi, synthesizer sederhana, sistem notifikasi suara, dan eksperimen digital signal processing, data audio perlu dikirim dari prosesor ke perangkat audio eksternal secara sinkron dan andal. Dalam konteks ini, I2S (Inter-IC Sound) menjadi protokol yang sangat relevan karena dirancang khusus untuk transmisi data audio digital antarchip.

I2S merupakan standar antarmuka serial audio yang dikembangkan oleh Philips dan banyak digunakan untuk menghubungkan prosesor, codec, DAC, dan perangkat audio digital lain [1]. Protokol ini menggunakan tiga sinyal utama, yaitu *bit clock* (BCLK), *word select* (WS/LRCLK), dan *serial data* (SD). Relasi timing antar sinyal tersebut harus dipenuhi dengan ketat agar penerima audio dapat merekonstruksi sampel digital dengan benar. Dukungan terhadap berbagai *sample rate* dan *bit depth* membuat I2S cocok untuk implementasi audio pada FPGA, terutama ketika dibutuhkan fleksibilitas desain pada level register transfer.

FPGA menawarkan keunggulan utama dalam pengembangan sistem audio digital karena logika perangkat kerasnya dapat dikustomisasi sesuai kebutuhan aplikasi [2]. Dibandingkan pendekatan berbasis perangkat keras tetap, FPGA memungkinkan perancangan antarmuka komunikasi, logika buffer, pembagi clock, dan jalur data serial secara langsung dalam Verilog atau VHDL [3]. Ketika dikombinasikan dengan soft processor seperti MicroBlaze V, FPGA juga memberikan model kerja hardware-software co-design: fungsi real-time dan timing kritis ditangani oleh perangkat keras, sementara konfigurasi, pengiriman data, dan pengujian dikendalikan oleh perangkat lunak [4].

MicroBlaze V sebagai prosesor soft-core berbasis RISC-V [5] pada ekosistem AMD/Xilinx [6] menyediakan fondasi yang sesuai untuk integrasi peripheral kustom pada Vivado dan Vitis. Melalui antarmuka AXI4-Lite [7], peripheral dapat diakses dengan model memory-mapped I/O yang sederhana namun efektif. Dengan pendekatan ini, prosesor dapat menulis sampel audio, mengatur mode operasi, serta membaca status dari peripheral I2S menggunakan instruksi load/store biasa. Pendekatan tersebut memudahkan pengembangan perangkat lunak uji sekaligus menjaga modularitas desain perangkat keras.

Berdasarkan kebutuhan tersebut, skripsi ini berfokus pada perancangan **IP core transmitter I2S berbasis Verilog** dengan antarmuka **AXI4-Lite** yang dapat diintegrasikan ke dalam SoC **MicroBlaze V** pada board **Basys3** [10]. Kode sumber dan dokumentasi proyek ini tersedia secara terbuka pada repositori GitHub¹. Target keluaran audio adalah modul **PCM5102A** sebagai DAC eksternal. Desain difokuskan pada mode **TX-only**, format **Philips I2S**, stereo, dengan **24 bit per saluran** dalam **slot 32 bit**, serta dua mode frekuensi sampling nominal sekitar 48 kHz dan 96 kHz. Penelitian ini mencakup perancangan RTL, integrasi Vivado Block Design [11], pengembangan perangkat lunak uji di Vitis [12], pengukuran sinyal, dan bukti keluaran audio.



Gambar 1.1. Ringkasan arsitektur sistem SoC dengan IP I2S TX AXI4-Lite dan DAC PCM5102A.

1.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, rumusan masalah penelitian ini adalah:

1. Bagaimana merancang **IP core transmitter I2S** berbasis Verilog dengan antarmuka

¹<https://github.com/adakhaddad/thes2is>

AXI4-Lite sehingga dapat dikontrol dari program Vitis melalui register memory-mapped?

2. Bagaimana mendukung **dua mode frekuensi sampling** (sekitar 48 kHz dan 96 kHz) dari satu `audio_clk` dengan pembagi yang dapat dipilih, sambil memperhatikan frekuensi aktual hasil Clocking Wizard?
3. Bagaimana merancang **FIFO TX** dan **interupsi watermark** untuk meminimalkan *underrun* dan mengurangi beban CPU saat pengisian data audio?
4. Bagaimana melakukan **verifikasi fungsional dan timing** melalui simulasi bila tersedia, pengukuran sinyal, dan uji keluaran audio pada PCM5102A?

1.3 Tujuan Penelitian

Tujuan penelitian ini disusun agar selaras dengan rumusan masalah yang telah ditetapkan.

1.3.1 Tujuan perancangan dan integrasi perangkat keras

1. Merancang dan mengimplementasikan modul **I2S TX** dengan **AXI4-Lite slave**, register kontrol dan data, serializer **Philips I2S**, serta **FIFO TX** untuk mendukung pengiriman sampel audio stereo.
2. Mengimplementasikan **pembagi clock** untuk dua mode frekuensi sampling nominal dan mengintegrasikan IP ke dalam **Vivado Block Design** pada FPGA **Basys3** beserta pemetaan pin pada file `.xdc`.

1.3.2 Tujuan integrasi SoC dan perangkat lunak

1. Membangun SoC berbasis **MicroBlaze V** dengan AXI interconnect, BRAM, Clocking Wizard, dan akses ke peripheral I2S melalui **Vitis**.
2. Mengembangkan aplikasi **Vitis** dalam bahasa C untuk konfigurasi register, **prefill FIFO**, penanganan **ISR** isi ulang FIFO, dan pemutaran sinyal uji audio.

1.3.3 Tujuan verifikasi dan validasi

1. Memverifikasi sistem melalui pengamatan sinyal `i2s_bclk`, `i2s_ws`, `i2s_data`, level FIFO, serta bukti keluaran audio pada PCM5102A.
2. Melakukan synthesis dan implementation pada FPGA Xilinx Artix-7 serta menganalisis *resource utilization* seperti LUT, FF, dan BRAM.

1.4 Batasan Penelitian

Batasan penelitian dalam pengembangan **IP core I2S TX AXI4-Lite** ini meliputi:

1.4.1 Batasan perangkat keras dan RTL

1. Modul I2S dirancang sebagai **transmitter (TX-only)**; tidak mencakup receiver atau mode full-duplex.
2. Format audio dibatasi pada **Philips I2S**, stereo, **24 bit** per saluran dalam **slot 32 bit**; tidak mencakup TDM, left-justified, right-justified, atau multi-channel.
3. Dukungan frekuensi sampling dibatasi pada **dua mode** nominal, yaitu sekitar 48 kHz dan 96 kHz, yang berasal dari satu sumber `audio_clk`.
4. Transfer data audio ke IP menggunakan **programmed I/O** berbasis register/FIFO dan tidak mencakup integrasi **DMA**.
5. Pengujian perangkat keras dilakukan pada board **Basys3** dengan modul **PCM5102A**; hasil pada platform lain dapat berbeda.

1.4.2 Batasan SoC dan perangkat lunak

1. Prosesor yang digunakan adalah **MicroBlaze V** pada ekosistem Vivado dan Vitis.
2. Lingkungan perangkat lunak yang digunakan adalah **bare-metal** tanpa RTOS atau Linux pada ruang lingkup utama.
3. UART digunakan untuk debug dan logging, bukan sebagai jalur streaming audio utama.

1.4.3 Batasan pengujian dan dokumentasi

1. Pengujian dibatasi pada ILA, osiloskop bila tersedia, dan uji dengar sederhana pada keluaran audio.
2. Dokumentasi disusun untuk kebutuhan skripsi dan tidak dimaksudkan sebagai data-sheet komersial lengkap.

1.5 Manfaat Penelitian

1.5.1 Manfaat akademik

1. Menghasilkan contoh perancangan **IP I2S TX AXI4-Lite** berbasis Verilog yang dapat digunakan sebagai acuan pembelajaran audio digital pada FPGA.
2. Menyediakan studi kasus hardware-software co-design pada FPGA yang melibatkan RTL, integrasi SoC, dan pemrograman embedded C.
3. Memberikan dokumentasi register map, strategi buffering, dan validasi sinyal I2S yang dapat digunakan kembali pada penelitian sejenis.

1.5.2 Manfaat praktis

1. Menyediakan dasar untuk proyek **audio output** berbasis FPGA seperti generator nada, alarm, dan sistem multimedia sederhana.
2. Menunjukkan pola desain **FIFO + watermark interrupt** yang dapat diadaptasi pada peripheral streaming lain.
3. Menjadi fondasi pengembangan lebih lanjut menuju sistem audio digital yang lebih kompleks, misalnya integrasi DMA atau dukungan Linux/ASoC.

1.5.3 Manfaat pengembangan keahlian

1. Memberikan pengalaman praktis dalam Verilog HDL, AXI4-Lite, integrasi Vivado Block Design, dan pengembangan aplikasi Vitis.
2. Mengembangkan kemampuan debugging hardware-software melalui ILA, pengukuran sinyal, dan analisis hasil implementasi.
3. Menjadi portofolio yang relevan untuk bidang FPGA, embedded systems, dan desain SoC.

1.6 Sistematika Penulisan

Bab I membahas pendahuluan yang mencakup latar belakang, rumusan masalah, tujuan, batasan, manfaat, dan sistematika penulisan.

Bab II memuat tinjauan pustaka dan dasar teori mengenai audio digital, protokol I2S, antarmuka AXI4-Lite, buffering FIFO, Clocking Wizard, MicroBlaze V, dan PCM5102A.

Bab III menyajikan metode penelitian yang mencakup alat dan bahan, metode verifikasi, serta alur tahapan tugas akhir secara sistematis.

Bab IV membahas perancangan dan implementasi yang merinci spesifikasi fungsional IP, arsitektur register map, integrasi pada Vivado Block Design, serta detail logika RTL (CDC dan serializer).

Bab V menyajikan hasil dan pembahasan yang mencakup pengujian sinyal digital, analisis akurasi frekuensi sampling, identifikasi bug firmware, serta verifikasi akhir sistem.

Bab VI berisi kesimpulan dan saran pengembangan.

BAB II

TINJAUAN PUSTAKA DAN DASAR TEORI

2.1 Tinjauan Pustaka

Bagian ini membahas penelitian terdahulu yang relevan dengan implementasi I2S pada FPGA, integrasi peripheral kustom pada SoC berbasis prosesor soft-core, serta penggunaan AXI4-Lite sebagai antarmuka register.

2.1.1 Penelitian tentang implementasi I2S pada FPGA

Implementasi antarmuka I2S pada FPGA telah dilaporkan dalam beberapa konteks. [13] merancang antarmuka I2S digital audio pada FPGA untuk menghubungkan prosesor dengan codec audio, menunjukkan bahwa protokol I2S dapat diimplementasikan dengan latensi rendah dan mendukung *sample rate* hingga 96 kHz. Penelitian tersebut menggunakan pendekatan RTL langsung tanpa antarmuka AXI, sehingga berbeda dari pendekatan berbasis register yang digunakan pada penelitian ini.

[14] mengintegrasikan blok logika I2S bersama *embedded logic analyzer* pada FPGA berbiaya rendah. Kontribusi penelitian tersebut terutama pada sisi instrumentasi, sedangkan fokus penelitian ini adalah pada desain peripheral yang dapat dikontrol dari perangkat lunak melalui antarmuka AXI4-Lite.

Secara umum, penelitian-penelitian tersebut mengkonfirmasi kelayakan implementasi I2S pada FPGA dan menjadi landasan pemilihan arsitektur serializer pada penelitian ini. Namun demikian, sebagian besar publikasi tersebut berhenti pada pembuktian bahwa sinyal I2S dapat dibangkitkan secara benar pada level RTL atau pada pengujian modul terisolasi. Penelitian ini mengambil lingkup yang lebih sistemik, yakni mengkaji bagaimana blok I2S yang sama dapat dikemas sebagai peripheral yang dapat diprogram, diintegrasikan ke lingkungan SoC, dan divalidasi melalui perangkat lunak nyata yang berjalan pada prosesor soft-core. Dengan demikian, perbedaan utama penelitian ini bukan hanya pada implementasi protokol, melainkan pada pendekatan *hardware-software co-design* yang menyatukan register map, integrasi bus, dan validasi end-to-end. Perspektif sistem semacam ini juga sejalan dengan pendekatan desain *embedded FPGA* yang menempatkan integrasi IP, perangkat lunak, dan alur toolchain sebagai satu kesatuan proses pengembangan [15].

2.1.2 Penelitian tentang prosesor soft-core dan integrasi SoC

Waterman dkk. [5] memperkenalkan arsitektur RISC-V sebagai ISA terbuka yang modular dan ekstensibel, yang kemudian mendasari pengembangan banyak prosesor soft-core termasuk MicroBlaze V pada ekosistem AMD/Xilinx [6]. Ketersediaan antarmuka

AXI4-Lite yang terintegrasi pada MicroBlaze V menjadi dasar pemilihan prosesor ini dalam penelitian ini.

Sklyarov dkk. [16] mengeksplorasi perancangan sistem SoC hierarkis berbasis bus AXI pada FPGA. Penelitian tersebut menunjukkan bahwa peripheral kustom dapat diintegrasikan ke dalam SoC melalui AXI interconnect dengan cara yang modular dan dapat diskalakan – pendekatan yang sama diterapkan dalam penelitian ini menggunakan Vivado IP Integrator.

Crockett dkk. [4] memberikan gambaran komprehensif mengenai hardware-software co-design pada FPGA berbasis prosesor, termasuk pola pengembangan peripheral AXI dan aplikasi bare-metal melalui SDK. Meskipun konteks platform berbeda (Zynq vs. Basys3), metodologi integrasi yang diulas relevan dengan pendekatan yang digunakan pada penelitian ini.

2.1.3 Penelitian tentang sinkronisasi antar-domain clock (CDC)

Dalam sistem digital yang melibatkan lebih dari satu domain clock – seperti IP I2S yang memiliki domain AXI (100 MHz) dan domain audio (~ 24 MHz) – sinkronisasi lintas domain clock (*Clock Domain Crossing*, CDC) menjadi aspek kritikal.

Ginosar [17] memberikan analisis mendalam tentang masalah metastabilitas pada sistem digital dan teknik sinkronisasi flip-flop ganda yang umum digunakan. Prinsip ini diterapkan langsung pada RTL penelitian ini melalui mekanisme sinkronisasi write-counter 4-bit dari domain AXI ke domain audio.

2.1.4 Gap penelitian dan kontribusi

Berdasarkan tinjauan di atas, dapat diidentifikasi beberapa celah penelitian:

1. Dokumentasi lengkap mengenai pengembangan **IP I2S TX custom** untuk **MicroBlaze V** yang mencakup RTL, register map, integrasi Vivado, aplikasi Vitis, validasi hardware, dan identifikasi bug firmware masih relatif terbatas.
2. Sebagian besar penelitian yang ada menggunakan platform prosesor yang lebih besar (Zynq, Cortex-A) sehingga contoh pada platform sederhana berbasis Basys3 memberikan nilai referensi tersendiri.
3. Analisis bug pada lapisan firmware – khususnya justifikasi bit sampel dan timing loop berbasis cycle counter – belum banyak didokumentasikan secara eksplisit dalam konteks I2S pada FPGA.

Jika diringkas, penelitian terdahulu telah menunjukkan tiga hal penting: I2S dapat diimplementasikan secara andal pada FPGA, peripheral kustom dapat dihubungkan ke SoC berbasis AXI, dan CDC merupakan aspek kritis pada sistem multi-clock. Akan tetapi,

dokumentasi yang menyatukan ketiganya pada satu alur yang lengkap masih terbatas. Keterbatasan inilah yang menjadi dasar penyusunan penelitian ini, sehingga hasil yang diharapkan bukan hanya berupa IP yang berfungsi, tetapi juga uraian langkah integrasi dan validasi yang dapat direplikasi.

Kontribusi penelitian ini adalah:

1. Merancang dan mengimplementasikan **IP core I2S transmitter** berbasis Verilog dengan antarmuka **AXI4-Lite** yang kompatibel dengan **MicroBlaze V**.
2. Menyediakan alur lengkap dari desain RTL, integrasi SoC, pengembangan perangkat lunak uji, hingga validasi pada hardware Basys3 dengan PCM5102A.
3. Mengidentifikasi dan mendokumentasikan tiga bug firmware (justifikasi bit, masking tanda, timing loop) beserta solusinya yang dapat menjadi referensi praktis.
4. Menyajikan analisis akurasi clock audio berbasis Clocking Wizard Artix-7 yang dapat digunakan kembali pada desain serupa.

2.2 Dasar Teori

2.2.1 Arsitektur RISC-V dan MicroBlaze V

RISC-V adalah instruksi set arsitektur (ISA) terbuka berbasis prinsip *Reduced Instruction Set Computer* (RISC) yang bersifat modular [5]. MicroBlaze V adalah implementasi prosesor soft-core dari AMD/Xilinx yang mengadopsi ISA RISC-V (RV32I) untuk dijalankan di atas fabrik FPGA [6].

2.2.1.1 Model Memory-Mapped I/O (MMIO)

Dalam arsitektur RISC-V, komunikasi dengan peripheral dilakukan melalui mekanisme *Memory-Mapped I/O* (MMIO). Pada model ini, register internal peripheral dipetakan ke dalam ruang alamat memori prosesor. Instruksi akses memori standar seperti *sw* (*store word*) dan *lw* (*load word*) digunakan untuk menulis dan membaca register peripheral tersebut. Keuntungan utama dari pendekatan ini adalah keseragaman akses antara memori data dan peripheral, sehingga tidak diperlukan instruksi khusus I/O.

2.2.1.2 Register Kontrol dan Status (CSR)

RISC-V menyediakan kumpulan *Control and Status Registers* (CSR) untuk manajemen sistem dan telemetri. Salah satu CSR yang penting dalam penelitian ini adalah *mcycle*, yang merupakan counter 64-bit yang menghitung jumlah siklus detak (*clock cycles*) sejak prosesor diaktifkan [18]. Register ini digunakan untuk mengimplementasikan *timing loop* presisi tinggi pada aplikasi perangkat lunak bare-metal.

2.2.2 Protokol Komunikasi AXI4-Lite

AXI4-Lite adalah varian sederhana dari protokol *Advanced eXtensible Interface* (AXI) yang dioptimalkan untuk komunikasi dengan peripheral berbasis register [7].

2.2.2.1 Kanal dan Sinyal AXI4-Lite

Protokol ini terdiri dari lima kanal transaksi independen:

1. **Write Address Channel (AW):** Mengirimkan alamat tujuan penulisan.
2. **Write Data Channel (W):** Mengirimkan data yang akan ditulis.
3. **Write Response Channel (B):** Memberikan status penyelesaian transaksi tulis.
4. **Read Address Channel (AR):** Mengirimkan alamat data yang akan dibaca.
5. **Read Data Channel (R):** Mengirimkan data yang dibaca beserta statusnya.

2.2.2.2 Mekanisme Handshake VALID/READY

Setiap transaksi pada AXI4-Lite diatur oleh mekanisme *handshake* dua arah menggunakan sinyal **VALID** dan **READY**. Sinyal **VALID** dibangkitkan oleh sumber (*source*) untuk menandakan bahwa informasi alamat atau data sudah valid. Sinyal **READY** dibangkitkan oleh tujuan (*destination*) untuk menandakan kesiapan menerima informasi tersebut. Transaksi dianggap terjadi secara tepat pada siklus detak di mana kedua sinyal **VALID** dan **READY** bernilai tinggi (logika 1). Mekanisme ini memungkinkan terjadinya *backpressure* jika peripheral sedang sibuk, sehingga mencegah hilangnya data selama proses komunikasi.

2.2.3 Protokol komunikasi I2S

I2S (*Inter-IC Sound*) adalah protokol serial sinkron yang dirancang oleh Philips untuk mengirimkan data audio digital antarperangkat [1]. I2S digunakan secara luas untuk menghubungkan prosesor, FPGA, codec, DAC, dan ADC audio.

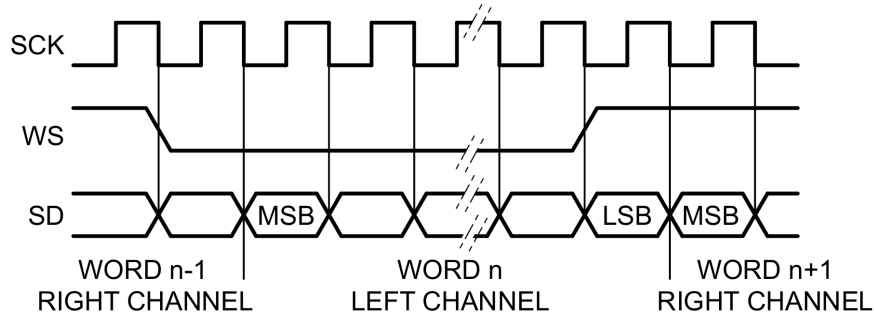
2.2.3.1 Sinyal I2S

I2S menggunakan tiga sinyal utama:

- **Serial Data (SD/DATA):** membawa data audio secara serial, MSB first.
- **Word Select (WS/LRCLK):** menandai kanal kiri (LOW) dan kanan (HIGH); frekuensinya sama dengan *sample rate*.
- **Bit Clock (BCLK/SCK):** clock serial yang menentukan perpindahan bit data.

Beberapa implementasi juga menggunakan sinyal **MCLK** (*Master Clock*) sebagai referensi frekuensi bagi perangkat penerima, meskipun banyak DAC modern seperti

PCM5102A dapat beroperasi tanpa MCLK menggunakan mode SCK [9].



Gambar 2.2. Diagram sinyal I2S Philips Standard (Sumber: NXP)

2.2.3.2 Format data Philips I2S

Pada format Philips I2S [1]:

- data dikirim **MSB first**;
- transisi WS terjadi **satu periode BCLK sebelum MSB** kanal berikutnya (*one-bit delay*);
- bit pertama setelah peralihan WS selalu bernilai nol (bit tunda wajib);
- data kiri dan kanan dikirim bergantian secara serial;
- implementasi praktis menggunakan slot 16, 24, atau 32 bit per kanal.

Pada penelitian ini, data audio dikirim sebagai **stereo 24 bit per saluran dalam slot 32 bit** sehingga satu frame stereo memerlukan 64 pulsa BCLK, dan lebar slot dapat dikonfigurasi melalui field `SAMPLE_WIDTH` pada register `CONTROL`. Pemilihan format 24-bit dalam slot 32-bit memberikan dua keuntungan praktis. Pertama, format ini umum dipakai oleh DAC audio modern sehingga kompatibilitas perangkat lebih mudah dipenuhi. Kedua, slot 32-bit menyederhanakan logika serializer dan pemetaan register 32-bit pada AXI4-Lite, karena satu transaksi tulis dapat langsung memuat satu sampel kanal tanpa perlu pemadatan bit tambahan di sisi perangkat lunak.

2.2.3.3 Relasi *sample rate*, BCLK, dan format slot

Frekuensi sampling F_s menentukan jumlah frame audio stereo per detik. Jika satu frame menggunakan N bit clock:

$$F_s = \frac{f_{\text{BCLK}}}{N} \quad (2-1)$$

Untuk format stereo 32-bit per kanal, $N = 64$. Hubungan antara clock master dan BCLK dinyatakan sebagai:

$$f_{\text{BCLK}} = \frac{f_{\text{MCLK}}}{\text{divider}} \quad (2-2)$$

Dengan $f_{\text{MCLK}} = 24,576$ MHz dan divider 8, diperoleh $f_{\text{BCLK}} = 3,072$ MHz dan $F_s = 48$ kHz.

2.2.4 DAC audio PCM5102A

PCM5102A adalah DAC audio stereo yang menerima data digital melalui antarmuka audio serial I2S [9]. Spesifikasi utama yang relevan:

- Mendukung *sample rate* hingga 384 kHz dan resolusi hingga 32 bit.
- Dapat beroperasi tanpa MCLK eksternal (mode SCK).
- Kompatibel dengan tegangan logika 3,3 V, cocok untuk antarmuka PMOD Basys3.
- Memiliki output analog stereo langsung tanpa filter eksternal tambahan.

Dalam konteks penelitian ini, PCM5102A berperan sebagai perangkat penerima data I2S dari FPGA. Keberhasilan sistem bergantung pada ketepatan timing BCLK, WS, dan DATA.

2.2.5 FPGA dan Basys3

FPGA (*Field-Programmable Gate Array*) adalah perangkat semikonduktor yang logika internalnya dapat dikonfigurasi ulang setelah fabrikasi [2]. Keunggulan FPGA untuk sistem audio digital mencakup kemampuan mengimplementasikan logika serial waktu nyata dengan latensi deterministik, pembangkitan clock presisi melalui PLL/MMCM internal, serta integrasi antara perangkat keras dan perangkat lunak melalui prosesor soft-core [19].

Basys3 adalah board FPGA berbasis Xilinx Artix-7 (XC7A35T) yang digunakan sebagai platform implementasi utama pada penelitian ini [10]. Board ini menyediakan osilator 100 MHz, antarmuka USB-UART, dan header PMOD untuk koneksi ke perangkat eksternal.

2.2.6 MicroBlaze V dan memory-mapped I/O

MicroBlaze V adalah prosesor soft-core berbasis RISC-V yang terintegrasi dalam ekosistem Vivado dan Vitis [6]. Peripheral kustom diakses melalui model **memory-mapped I/O**: register peripheral dipetakan ke alamat tertentu dalam ruang alamat prosesor, sehingga dapat dibaca dan ditulis menggunakan instruksi `load/store` biasa.

Dengan pendekatan ini, perangkat lunak dapat:

- menulis register `CONTROL` untuk mengaktifkan peripheral dan mengatur mode;
- mengisi register `DATA_LEFT/DATA_RIGHT` dengan sampel audio;
- membaca register untuk verifikasi *readback*.

Pada penelitian ini, keberadaan prosesor soft-core tidak hanya berfungsi sebagai penghasil sampel audio, tetapi juga sebagai sarana untuk mengevaluasi apakah peripheral benar-benar mudah dikendalikan dari perangkat lunak. Dengan kata lain, keberhasilan desain tidak cukup diukur dari sinyal I2S yang benar di simulator, tetapi juga dari kemampuan sistem untuk menerima konfigurasi, memproses perubahan mode, dan mempertahankan keluaran audio yang stabil saat dikendalikan melalui *memory-mapped I/O*.

RISC-V menyediakan *Control and Status Register* (CSR) `mcycle` yang merupakan counter siklus CPU 64-bit bebas-jalan (*free-running*) [18]. Register ini diakses melalui instruksi `rdcycle/rdcycleh` dan digunakan pada penelitian ini untuk implementasi timing loop dengan presisi 10 ns (pada clock 100 MHz).

2.2.7 Antarmuka AXI4-Lite

AXI4-Lite adalah subset sederhana dari protokol AMBA AXI4 yang ditujukan untuk peripheral berbasis register [7]. Karakteristik utama:

- Lima kanal independen: Write Address (AW), Write Data (W), Write Response (B), Read Address (AR), dan Read Data (R).
- Protokol handshake `VALID/READY` pada setiap kanal memungkinkan *backpressure* tanpa kehilangan data.
- Tidak mendukung *burst transfer* – setiap transaksi membaca atau menulis satu word 32-bit.
- Cocok untuk konfigurasi register peripheral dengan kebutuhan bandwidth rendah.

Dokumentasi implementasi AXI4-Lite pada ekosistem Xilinx tersedia pada [20]. AXI4-Lite dipilih dalam penelitian ini karena karakter peripheral yang dirancang lebih dekat ke pola konfigurasi register dibanding *streaming* berkecepatan tinggi. Register `CONTROL` digunakan untuk mengubah mode kerja, sedangkan `DATA_LEFT` dan `DATA_RIGHT` digunakan untuk memuat sampel saat validasi. Pendekatan ini menyederhanakan integrasi pada Vivado IP Integrator dan memudahkan penelusuran bug, meskipun konsekuensinya adalah throughput sistem tetap bergantung pada kecepatan CPU saat menulis sampel secara *programmed I/O*.

2.2.8 Sinkronisasi lintas domain clock (CDC)

Sistem I2S melibatkan dua domain clock yang tidak sinkron: domain AXI (100 MHz) dan domain audio (≈ 24 MHz). Data yang melintas antara dua domain dapat mengalami metastabilitas jika tidak disinkronkan dengan benar [17].

Teknik umum yang digunakan adalah **sinkronisasi flip-flop ganda**: sinyal dari domain sumber disampling dua kali berurutan oleh clock domain tujuan sebelum digu-

nakan. Hal ini memberikan waktu yang cukup bagi flip-flop untuk keluar dari kondisi metastabil.

Pada penelitian ini, RTL menggunakan **write-counter 4-bit** yang bertambah pada setiap transaksi AXI write. Domain audio menyinkronkan counter ini dengan dua flip-flop dan mendeteksi setiap perubahan, sehingga setiap penulisan register dari CPU dijamin terdeteksi tanpa terlewat. Pemilihan clock audio melalui BUFGMUX juga disinkronkan terhadap domain AXI sebelum digunakan sebagai sinyal selektor. Pendekatan ini dipilih karena yang perlu dipindahkan antar-domain pada penelitian ini bukan aliran data lebar berkecepatan tinggi, melainkan peristiwa pembaruan register yang relatif jarang dan terstruktur. Oleh sebab itu, kombinasi write-counter, sinkronisasi dua flip-flop, dan *shadow register* pada batas frame sudah memadai untuk menjaga koherensi data tanpa kompleksitas FIFO asinkron penuh.

2.2.9 Clocking Wizard dan frekuensi aktual

Pada FPGA Xilinx, *Clocking Wizard* memanfaatkan MMCM (*Mixed-Mode Clock Manager*) untuk menghasilkan clock internal yang mendekati frekuensi target [8]. Nilai yang dihasilkan (*actual frequency*) umumnya sedikit berbeda dari nilai nominal karena keterbatasan rasio frekuensi MMCM. Selisih ini berpengaruh langsung pada F_s aktual dan perlu dicatat saat evaluasi hasil.

2.2.10 Verilog HDL dan perancangan RTL

Verilog digunakan untuk mendeskripsikan logika perangkat keras pada level RTL [3]. Pada penelitian ini, Verilog digunakan untuk membangun:

- antarmuka slave AXI4-Lite dengan mesin state tulis dan baca;
- mekanisme CDC berbasis write-counter;
- logika pemilih clock (BUFGMUX);
- serializer I2S sesuai spesifikasi Philips;
- logika *power-on reset* domain audio.

Prinsip desain digital yang digunakan merujuk pada [21] dan [22].

2.2.11 Perbandingan dengan IP I2S bawaan AMD/Xilinx

Sebagai pembanding, AMD menyediakan *LogiCORE* I2S Transmitter dan Receiver untuk integrasi cepat di Vivado [23]. IP bawaan ini mendukung antarmuka AXI4-Stream, hingga empat kanal I2S, lebar data 16/24-bit, mode master, kedalaman FIFO yang dapat dikonfigurasi, ekstraksi dan penyisipan status kanal AES, format left/right justified, serta opsi LRCLK 32-bit. Secara umum, fitur tersebut ditujukan untuk aplikasi

audio yang berorientasi aliran data dan integrasi sistem yang luas.

Ruang lingkup penelitian ini berbeda karena IP yang dikembangkan berfokus pada **TX-only** dengan antarmuka **AXI4-Lite** berbasis register memory-mapped. Pendekatan ini dipilih agar pengendalian dari MicroBlaze V tetap sederhana tanpa DMA, sekaligus membuat mekanisme CDC, pembagian clock, dan pembentukan frame I2S lebih transparan pada level RTL. Dengan demikian, tujuan penelitian bukan menggantikan IP bawaan AMD/Xilinx, melainkan menyediakan alternatif yang lebih kecil, terukur, dan sesuai untuk validasi akademik.

2.2.12 Analisis pemilihan metode

Metode yang dipilih pada penelitian ini didasarkan pada beberapa pertimbangan:

1. **MicroBlaze V** dipilih karena terintegrasi dengan baik pada ekosistem Vivado dan Vitis serta sesuai untuk sistem embedded bare-metal [6].
2. **AXI4-Lite** dipilih karena sesuai untuk peripheral berbasis register dengan konfigurasi sederhana dan didukung penuh oleh IP Integrator [20].
3. **Verilog HDL** dipilih karena umum digunakan untuk desain RTL pada FPGA dan memiliki dukungan sintesis yang baik pada Vivado [3].
4. Pendekatan **hardware-software co-design** dipilih agar peripheral dapat divalidasi tidak hanya dari sisi RTL, tetapi juga dari sisi penggunaan nyata oleh perangkat lunak [4].

Selain pertimbangan di atas, lingkup penelitian sengaja dibatasi pada jalur **TX-only** tanpa FIFO dan tanpa DMA. Pembatasan ini dilakukan agar fokus penelitian tetap berada pada ketepatan protokol I2S, keamanan CDC, dan integrasi peripheral kustom ke SoC berbasis MicroBlaze V. Konsekuensinya, sistem yang dihasilkan belum ditujukan untuk playback audio berdurasi panjang dengan throughput tinggi, melainkan sebagai platform validasi yang menekankan kejelasan arsitektur, kemudahan pengendalian, dan keterlancaran bug. Pertimbangan ini konsisten dengan literatur desain sistem FPGA embedded yang menekankan pentingnya memilih arsitektur yang cukup sederhana untuk divalidasi menyeluruh sebelum diperluas ke fitur throughput yang lebih tinggi [15].

BAB III

METODE PENELITIAN

3.1 Alat dan Bahan

3.1.1 Perangkat keras

3.1.1.1 Board FPGA

Basys3 digunakan sebagai platform implementasi utama [10]. Board ini berbasis FPGA Xilinx Artix-7 dan menyediakan sumber clock 100 MHz, antarmuka USB-UART, serta header PMOD untuk koneksi ke perangkat eksternal.

3.1.1.2 Modul DAC audio

PCM5102A Audio DAC Module digunakan sebagai penerima data I2S [9]. Modul ini mendukung antarmuka audio serial, berbagai *sample rate* hingga 384 kHz, dan keluaran analog stereo untuk verifikasi suara.

3.1.1.3 Perangkat bantu

- **Oscilloscope / logic analyzer / ILA** untuk memverifikasi sinyal BCLK, WS, dan DATA.
- **Speaker atau headphone aktif** untuk uji keluaran audio.
- **Breadboard dan kabel jumper** untuk koneksi Basys3 ke PCM5102A.

3.1.2 Perangkat lunak

- **AMD Vivado Design Suite** untuk RTL design, synthesis, implementation, dan integrasi SoC [11].
- **Xilinx Vitis** untuk pengembangan perangkat lunak uji dalam bahasa C [12].
- **Vivado Simulator** atau simulasi lain bila digunakan untuk verifikasi RTL.
- **LaTeX** untuk penulisan dokumen skripsi.

3.2 Metode yang Digunakan

3.2.1 Metode perancangan hardware

Perancangan perangkat keras dilakukan dengan tahapan berikut:

1. Menentukan spesifikasi I2S TX meliputi format data [1], lebar sampel, mode frekuensi sampling, dan pemetaan register.

2. Mendesain arsitektur modul yang mencakup AXI4-Lite slave [7], register data dan kontrol, pembagi clock, serta serializer I2S.
3. Mengimplementasikan RTL dalam Verilog [3] dengan pemisahan blok sekuensial dan kombinasional secara jelas.
4. Mengintegrasikan peripheral ke dalam Vivado IP Integrator [11] bersama MicroBlaze V, BRAM, clocking, dan peripheral pendukung.

Alur tersebut sejalan dengan pendekatan pengembangan sistem FPGA embedded yang menempatkan definisi spesifikasi, implementasi RTL, integrasi IP, dan validasi perangkat lunak sebagai tahapan yang saling terkait, bukan pekerjaan yang berdiri sendiri [15].

3.2.2 Metode verifikasi dan simulasi

Verifikasi dilakukan pada dua level:

1. **Validasi hardware**, untuk memastikan desain tetap bekerja setelah *synthesis* dan *implementation* pada FPGA nyata.
2. **Verifikasi fungsional RTL**, untuk mereproduksi perilaku sinyal I2S, urutan bit, dan logika DATA/CONTROL secara terkontrol melalui simulasi.

Parameter yang diamati meliputi:

- relasi timing `i2s_bclk`, `i2s_ws`, dan `i2s_data`;
- kontinuitas aliran sampel pada skenario uji;
- akurasi frekuensi sampling nominal rendah dan tinggi;
- keberhasilan keluaran audio pada DAC.

Suatu pengujian dinyatakan lulus apabila empat syarat utama terpenuhi, yaitu: struktur frame terdiri dari 64 pulsa BCLK per sampel stereo, transisi WS terjadi satu BCLK sebelum MSB kanal berikutnya, bit tunda setelah transisi WS bernilai nol, dan DAC mampu menghasilkan audio yang konsisten dengan mode kanal serta frekuensi yang dipilih. Dengan definisi ini, verifikasi tidak berhenti pada kecocokan waveform semata, tetapi juga menilai apakah perilaku sistem sudah sesuai dengan tujuan operasional peripheral.

Pada alur kerja aktual penelitian ini, verifikasi perilaku dasar terlebih dahulu dilakukan melalui *bitstream bring-up*, pengamatan ILA, dan pengukuran pada perangkat keras. Setelah hubungan timing utama telah teridentifikasi pada FPGA nyata, dibuat *testbench* Vivado/XSim pada proyek `C:\tesis2s\testblock` untuk mereproduksi kondisi `enable`, `mute`, pemilihan `fs_mode`, dan serialisasi sampel kiri-kanan secara lebih formal. Dengan demikian, simulasi pada penelitian ini berfungsi sebagai verifikasi pelengkap yang mendokumentasikan ulang perilaku yang telah diamati pada perangkat keras, bukan sebagai satu-satunya sumber validasi.

3.2.3 Metode integrasi SoC

Integrasi peripheral ke dalam SoC dilakukan melalui alur berikut:

1. **IP Packaging:** mengemas modul I2S sebagai custom IP menggunakan Vivado IP Integrator [11].
2. **Block Design:** menghubungkan MicroBlaze V [6], AXI interconnect [20], BRAM, Clocking Wizard [8], UART, dan IP I2S.
3. **Address Allocation:** menetapkan base address peripheral agar dapat diakses dari Vitis [12].
4. **Constraint Management:** memetakan pin I2S ke konektor board sesuai file `.xdc` [10].

3.2.4 Metode pengembangan software

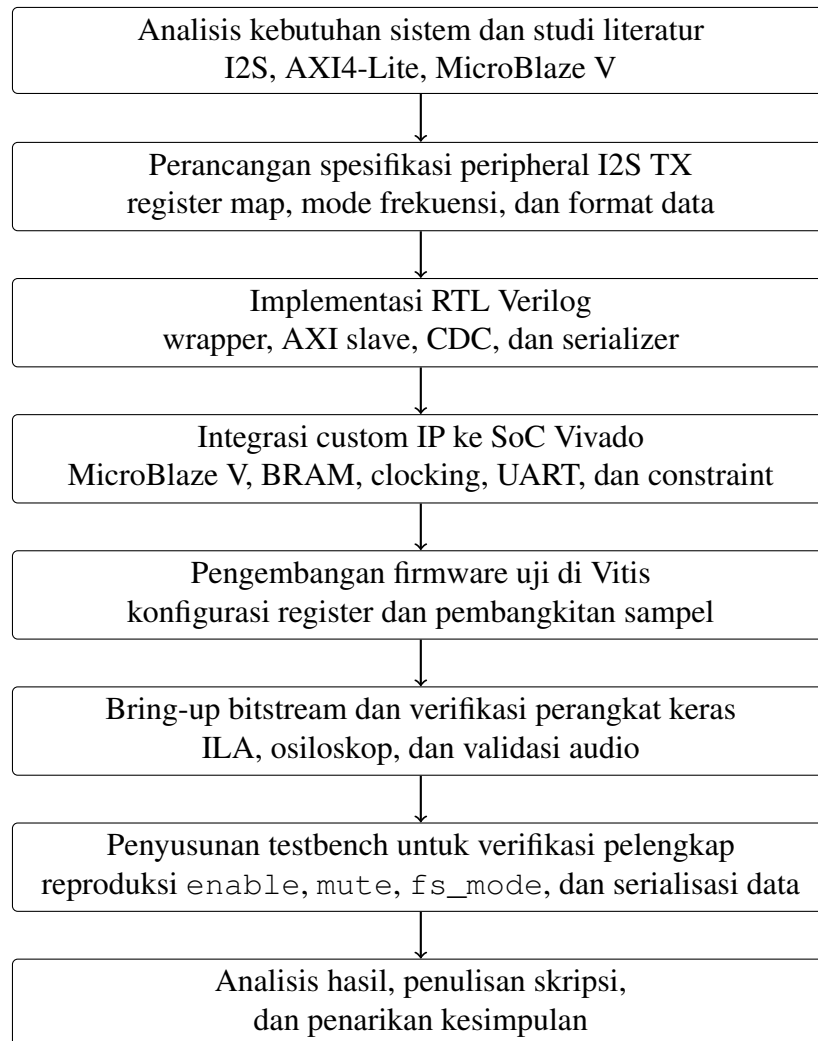
Perangkat lunak uji dikembangkan dalam bahasa C pada Vitis dengan fungsi utama:

- menginisialisasi register peripheral I2S;
- menulis sampel audio ke register `DATA_LEFT` dan `DATA_RIGHT`;
- mengaktifkan mode operasi dan memilih frekuensi sampling;
- menyinkronkan waktu penulisan data ke dalam register secara langsung (*programmed I/O*);
- menghasilkan sinyal uji sederhana seperti gelombang sinus atau *square wave*.

3.3 Alur Tugas Akhir

Penelitian ini mengikuti tahapan berikut:

1. Analisis kebutuhan sistem audio digital dan studi literatur tentang I2S, AXI4-Lite, dan MicroBlaze V.
2. Perancangan spesifikasi peripheral I2S TX.
3. Implementasi RTL Verilog dan verifikasi awal.
4. Integrasi ke dalam SoC pada Vivado Block Design.
5. Pengembangan perangkat lunak uji di Vitis.
6. Pengujian hardware dengan ILA, pengukuran sinyal, dan validasi audio.
7. Analisis hasil, penyusunan dokumen, dan penarikan kesimpulan.



Gambar 3.3. Diagram alir tahapan penelitian tugas akhir.

BAB IV

PERANCANGAN DAN IMPLEMENTASI

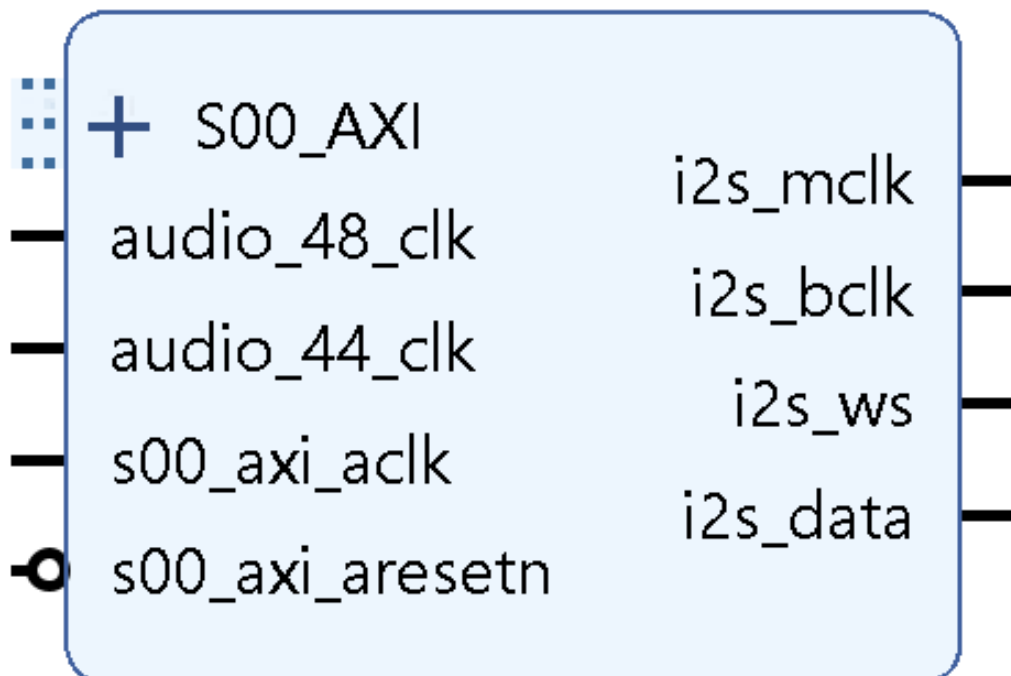
4.1 Spesifikasi dan arsitektur IP I2S TX

Bagian ini merangkum **perancangan fungsional** IP yang menjadi inti skripsi. Implementasi difokuskan pada peripheral **I2S transmitter** dengan antarmuka **AXI4-Lite** agar dapat dikontrol oleh program C pada MicroBlaze V [6].

4.1.1 Daftar fitur IP

- Antarmuka **AXI4-Lite** 32 bit untuk konfigurasi dan pengisian data audio [7].
- Mode **TX-only**, stereo, format **Philips I2S** [1], **24 bit** per saluran dalam **slot 32 bit**.
- Dukungan dua keluarga frekuensi sampling nominal (**48/96 kHz** dan **44,1/88,2 kHz**), melalui pembagi clock internal dan BUFGMUX [8].
- **Jalur data** lewat register `DATA_LEFT/DATA_RIGHT` dan `CONTROL`, **tanpa FIFO** internal.

4.1.2 Blok diagram perancangan IP



Gambar 4.4. Blok diagram internal IP I2S TX.

4.1.3 Register map

Tabel 4.1 merangkum register word-aligned yang digunakan pada implementasi final.

Tabel 4.1. Register map IP I2S AXI4-Lite.

Offset	Nama Register	Fungsi	Packing / Catatan
0x00	DATA_LEFT	Sampel kanal kiri	MSB di bit [SAMPLE_WIDTH-1]
0x04	DATA_RIGHT	Sampel kanal kanan	MSB di bit [SAMPLE_WIDTH-1]
0x08	CONTROL	Konfigurasi peripheral	Lihat Tabel 4.2

Tabel 4.2. Bitfield register CONTROL.

Bit(s)	Field	Default	Fungsi / Keterangan
0	ENABLE	0	1 = output audio aktif, 0 = output nonaktif
1	MUTE	0	1 = data audio dipaksa nol, clock tetap berjalan
2	FS_FAMILY	0	0 = keluarga 48/96 kHz, 1 = keluarga 44,1/88,2 kHz
3	FS_MODE	0	0 = Fs rendah (48/44,1 kHz); 1 = Fs tinggi (96/88,2 kHz)
12 : 8	SAMPLE_WIDTH	32	Lebar bit sampel per kanal (1–32)

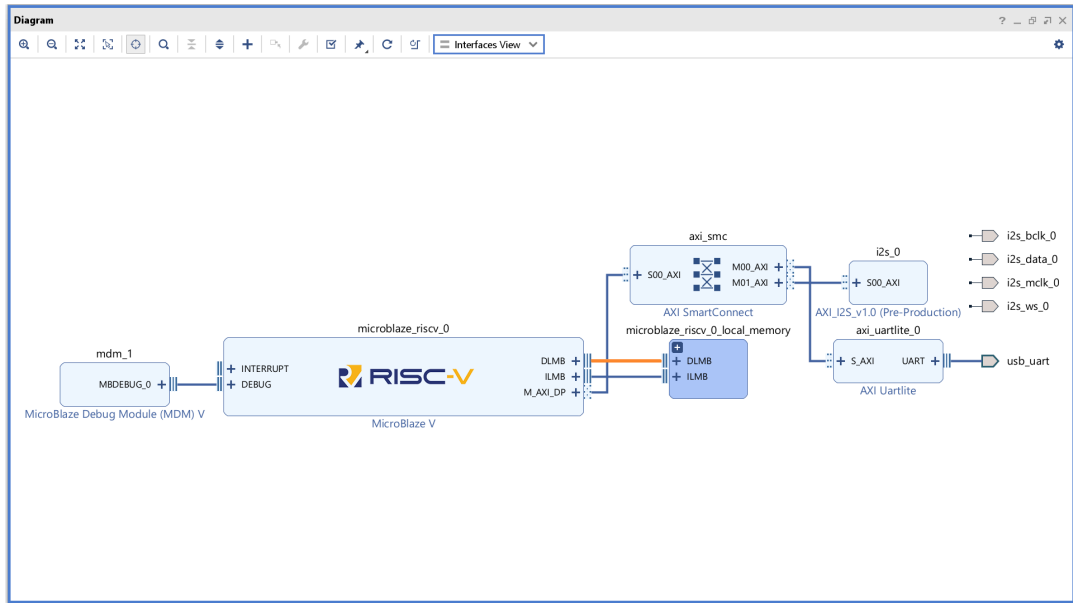
4.2 Implementasi RTL dan Integrasi Vivado

RTL direalisasikan dalam dua berkas Verilog: `i2s.v` sebagai *top-level wrapper* dan `i2s_slave_lite_v1_0_S00_AXI.v` sebagai modul utama yang mencakup seluruh logika AXI4-Lite, CDC, dan serializer I2S.

4.2.1 Pemetaan pin (constraint)

Pin I2S dihubungkan ke header PMOD JA pada Basys3 melalui file `cnstr.xdc` sebagai berikut:

```
set_property -dict {PACKAGE_PIN J1 IOSTANDARD LVCMOS33} [get_ports i2s]
set_property -dict {PACKAGE_PIN L2 IOSTANDARD LVCMOS33} [get_ports i2s]
set_property -dict {PACKAGE_PIN J2 IOSTANDARD LVCMOS33} [get_ports i2s]
set_property -dict {PACKAGE_PIN G2 IOSTANDARD LVCMOS33} [get_ports i2s]
```



Gambar 4.5. Block Design Vivado: MicroBlaze V terhubung ke IP I2S melalui AXI SmartConnect.

4.2.2 Detail logika RTL (serializer dan CDC)

4.2.2.1 Mekanisme Clock Domain Crossing (CDC)

Data audio ditulis oleh MicroBlaze V pada domain `S_AXI_ACLK` (100 MHz), sementara serializer beroperasi pada domain `internal_mclk` (~24 MHz). Sinkronisasi dilakukan menggunakan flip-flop ganda dan latching pada batas frame stereo.

```

1 always @(posedge dest_clk) begin
2     stage1_q <= src_in;
3     stage2_q <= stage1_q;
4 end

```

Listing IV.1. Cuplikan modul `cdc_sync` pada RTL.

4.2.2.2 Logika serializer Philips I2S

```

1 case (bit_count_q)
2     6'd0, 6'd32: i2s_next_serial_bit = 1'b0; // delay bit
3     default: begin
4         // Serialisasi MSB first
5     end
6 endcase

```

Listing IV.2. Cuplikan logika serializer I2S.

4.3 Konfigurasi Clocking Wizard

Clocking Wizard [8] dikonfigurasi menghasilkan tiga keluaran dari satu osilator 100 MHz:

1. `clk_out1` = 100 MHz (AXI ACLK).
2. `clk_out2` = 24,576 MHz (`audio_48_clk`).
3. `clk_out3` = 22,579 MHz (`audio_44_clk`).

Tabel 4.3. Konfigurasi clock hasil Clocking Wizard.

Clock	Target	Aktual
<code>clk_out1</code>	100,000 MHz	100,00000 MHz
<code>clk_out2</code>	24,576 MHz	24,59677 MHz
<code>clk_out3</code>	22,579 MHz	22,42647 MHz

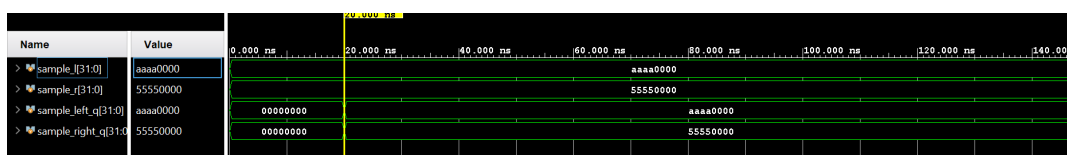
BAB V

HASIL DAN PEMBAHASAN

Bab ini menyajikan hasil pengujian sinyal, analisis frekuensi sampling, serta identifikasi dan perbaikan bug firmware yang ditemukan selama validasi perangkat keras.

5.1 Pengujian Sinyal

Pengamatan sinyal pada `i2s_bclk`, `i2s_ws`, dan `i2s_data` dilakukan menggunakan alat ukur digital (osiloskop/logic analyzer) untuk memverifikasi timing Philips I2S [1].



Gambar 5.6. Tangkapan waveform: BCLK, WS, dan DATA – satu frame stereo 64 BCLK.

Verifikasi ILA mengkonfirmasi:

- Satu frame stereo terdiri dari tepat 64 periode BCLK (32 kiri + 32 kanan).
- WS beralih satu BCLK *sebelum* MSB data, sesuai spesifikasi Philips.
- Bit pertama setelah setiap peralihan WS selalu bernilai 0.

5.2 Analisis Frekuensi Sampling

Tabel 5.1. Hasil pengujian clock I2S pada keluarga frekuensi 48 kHz.

Mode	MCLK	Divider	BCLK	F_s
F _S _MODE = 0	24,597 MHz	÷8	3,0746 MHz	48,03 kHz
F _S _MODE = 1	24,597 MHz	÷4	6,1493 MHz	96,08 kHz

5.3 Identifikasi dan Perbaikan Bug Firmware

Selama validasi perangkat keras ditemukan beberapa bug yang bersumber dari **kode firmware** (`helloworld.c`), bukan dari RTL.

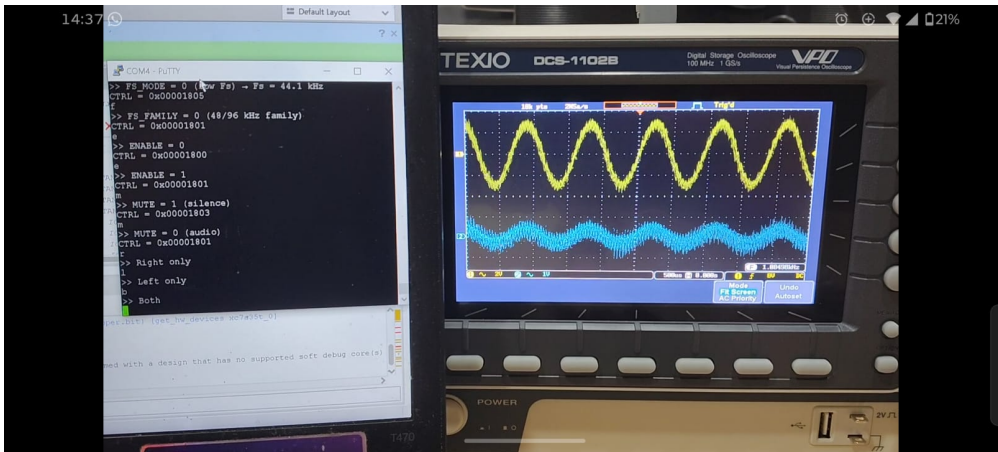
5.3.1 Bug 1: Masker Merusak Bit Tanda Sampel

Gejala: Amplitudo gelombang tidak seimbang, waveform terdistorsi. **Solusi:** Menghapus penggunaan masker bit dan menyesuaikan geseran agar MSB int16 tepat di bit `[g_width-1]`.

5.3.2 Bug 2: Timing Loop Berbasis usleep()

Gejala: Frekuensi audio terukur lebih rendah dari target. **Solusi:** Mengganti `usleep()` dengan loop berbasis *deadline* menggunakan CSR `mcycle` RISC-V untuk presisi 10 ns.

5.4 Verifikasi Setelah Perbaikan



Gambar 5.7. Keluaran audio setelah perbaikan: kanal L dan R identik.

Tabel 5.2. Hasil verifikasi firmware setelah perbaikan bug.

Skenario Uji	Hasil yang Diharapkan	Status
Both + 1 kHz	L dan R identik, 1 kHz	✓
Left only + 1 kHz	L aktif, R = 0	✓
MUTE aktif	Kedua kanal = 0, clock tetap	✓

5.5 Pembahasan terhadap Tujuan Penelitian

Hasil penelitian menunjukkan bahwa pendekatan register-mapped AXI4-Lite efektif untuk membangun peripheral I2S TX. Seluruh perbaikan bug dilakukan pada lapisan firmware, yang mengonfirmasi bahwa desain RTL sudah stabil dan benar sejak awal.

5.6 Perbandingan dengan referensi desain

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 1.251 ns	Worst Hold Slack (WHS): 0.039 ns	Worst Pulse Width Slack (WPWS): 3.000 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 7290	Total Number of Endpoints: 7290	Total Number of Endpoints: 2637

All user specified timing constraints are met.

Gambar 5.8. Ringkasan timing desain.

BAB VI

KESIMPULAN DAN SARAN

6.1 Kesimpulan

Penelitian ini merancang dan mengimplementasikan **IP core transmitter I2S** [1] dengan antarmuka **AXI4-Lite** [7] pada SoC **MicroBlaze V** [6], board **Basys3** [10], dan keluaran menuju **PCM5102A** [9]. Berdasarkan hasil pada Bab IV, dapat disimpulkan:

1. IP berhasil diintegrasikan sebagai peripheral AXI4-Lite dengan peta register aktif `DATA_LEFT` (`0x00`), `DATA_RIGHT` (`0x04`), dan `CONTROL` (`0x08`). Register `0x0C` disediakan sebagai cadangan dan berhasil digunakan untuk *readback test* AXI yang memverifikasi koneksi bus.
2. Aplikasi Vitis bare-metal dapat mengendalikan IP melalui *memory-mapped I/O* (`Xil_Out32/Xil_In32`) untuk inisialisasi register `CONTROL`, pengisian sampel kanal kiri-kanan, dan verifikasi *readback*.
3. Pengujian sinyal mengkonfirmasi sinyal `BCLK` (3,07 MHz), `WS` (48,03 kHz), dan `DATA` mengikuti mekanisme **Philips I2S** [1]: 1-bit delay setelah setiap peralihan `WS`, 64 `BCLK` per frame stereo, dan `MSB-first` per kanal. Nilai aktual sesuai dengan analisis clock Clocking Wizard [8] (24,597 MHz).
4. Validasi audio pada **PCM5102A** [9] berhasil menghasilkan keluaran suara yang dapat didengar. Tiga bug firmware yang ditemukan selama validasi — yaitu *masker bit yang merusak tanda sampel*, *justifikasi sampel 32-bit yang tidak tepat*, dan *timing loop berbasis usleep yang tidak akumulatif* — berhasil diidentifikasi dan diperbaiki seluruhnya di sisi firmware, tanpa perubahan pada RTL.
5. RTL `i2s_slave_lite_v1_0_S00_AXI.v` terbukti benar secara fungsional sejak awal, mencakup mekanisme CDC berbasis write-counter yang robust [17], pemilihan clock melalui `BUFGMUX`, dan serializer Philips I2S yang sesuai spesifikasi [1].

6.2 Saran

1. Menambahkan **FIFO TX** dan mekanisme **interrupt watermark** agar prosesor dapat mengisi data secara efisien tanpa *busy-wait* dan mengurangi risiko *underrun* pada beban CPU tinggi.
2. Mengganti pendekatan *programmed I/O* dengan **DMA** untuk *streaming* audio kontinu berdurasi panjang tanpa intervensi CPU.
3. Menambahkan register **STATUS** yang dapat dibaca oleh firmware untuk memantau

kondisi *underrun*, level buffer, dan kesiapan latch CDC — memudahkan transisi ke driver tingkat sistem operasi.

4. Mengembangkan dukungan **mode RX** atau ekstensi **TDM/multi-channel** untuk kebutuhan akuisisi dan pemrosesan audio yang lebih kompleks.
5. Melakukan karakterisasi lanjutan terhadap akurasi frekuensi audio pada berbagai kondisi suhu dan beban, serta mempertimbangkan penggunaan **PLL audio eksternal** jika akurasi clock $\ll$ 100 ppm diperlukan.
6. Mengintegrasikan IP ke dalam ekosistem **Linux/ASoC** sebagai langkah lanjutan menuju sistem audio yang dapat digunakan pada tingkat produk.

DAFTAR PUSTAKA

- [1] Philips Semiconductors, “I²S bus specification,” Philips Semiconductors, Tech. Rep., Feb. 1986, revision 1. Tersedia: <https://www.nxp.com/docs/en/user-guide/UM11732.pdf>.
- [2] S. M. Trimberger, “Three ages of FPGAs: A retrospective on the first thirty years of FPGA technology,” *Proceedings of the IEEE*, vol. 103, no. 3, pp. 318–331, 2015.
- [3] S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed. Prentice Hall, 2003.
- [4] L. H. Crockett, R. A. Elliot, M. A. Enderwitz, and R. W. Stewart, “The Zynq book: Embedded processing with the ARM cortex-A9 on the Xilinx Zynq-7000 all programmable SoC,” *Strathclyde Academic Media*, 2014, iISBN 978-0992978709. Tersedia: <http://www.zynqbook.com>.
- [5] A. Waterman, Y. Lee, R. Avizienis, D. A. Patterson, and K. Asanović, “The RISC-V instruction set architecture,” in *Hot Chips 28 Symposium (HCS)*, 2016, pp. 1–52.
- [6] AMD Xilinx, *MicroBlaze Processor Reference Guide*, AMD Xilinx, 2021, tersedia: https://www.xilinx.com/support/documentation/sw_manuals/xilinx2021_2/ug984-vivado-microblaze-ref.pdf.
- [7] ARM Limited, “AMBA AXI and ACE protocol specification,” ARM Limited, Tech. Rep. ARM IHI 0022D, 2010, issue D. Tersedia: <https://developer.arm.com/documentation/ih0022/d/>.
- [8] AMD Xilinx, *Clocking Wizard v6.0 Product Guide*, AMD Xilinx, 2021, tersedia: https://www.xilinx.com/support/documentation/ip_documentation/clk_wiz/v6_0/pg065-clk-wiz.pdf.
- [9] Texas Instruments, “PCM5102A 2-VRMS, 112-dB audio stereo DAC with PLL and 32-bit, 384-kHz PCM interface,” Texas Instruments, Tech. Rep. SLAS764B, 2014, datasheet. Tersedia: <https://www.ti.com/lit/ds/symlink/pcm5102a.pdf>.
- [10] Digilent Inc., *Basys 3 FPGA Board Reference Manual*, Digilent Inc., 2020, tersedia: <https://digilent.com/reference/programmable-logic/basys-3/reference-manual>.
- [11] AMD Xilinx, *Vivado Design Suite User Guide: Designing with IP*, AMD Xilinx, 2021, tersedia: https://www.xilinx.com/support/documentation/sw_manuals/xilinx2021_2/ug896-vivado-ip.pdf.
- [12] —, *Vitis Unified Software Platform Documentation: Embedded Software Development*, AMD Xilinx, 2021, tersedia: https://www.xilinx.com/support/documentation/sw_manuals/xilinx2021_2/ug1400-vitis-embedded.pdf.
- [13] J. Sang, D. Peng, and M. Zhou, “Design and implementation of I2S digital audio interface on FPGA,” *Journal of Physics: Conference Series*, vol. 1601, no. 5, p. 052040, 2020.

- [14] A. Kaviani and D. Lewis, “Embedded logic analyzer and I2S audio interface on low-cost FPGA,” in *Proceedings of the IEEE International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2017, pp. 221–224.
- [15] A. Lafosse and D. Gaffe, *Embedded FPGA Systems: Design and Programming with Vivado*. Wiley-ISTE, 2018.
- [16] V. Sklyarov, I. Sklyarova, A. Barkalov, and L. Skliarova, “Hierarchical AXI-bus based SoC for FPGA,” in *2014 24th International Conference on Field Programmable Logic and Applications (FPL)*, 2014, pp. 1–4.
- [17] R. Ginosar, “Metastability and synchronizers: A tutorial,” *IEEE Design & Test of Computers*, vol. 28, no. 5, pp. 23–35, 2011.
- [18] A. Waterman and K. Asanović, “The RISC-V instruction set manual, volume I: Unprivileged ISA,” RISC-V Foundation, Tech. Rep. Document Version 20191213, 2019, tersedia: <https://github.com/riscv/riscv-isa-manual/releases/>.
- [19] I. Kuon, R. Tessier, and J. Rose, “FPGA architecture: Survey and challenges,” *Foundations and Trends in Electronic Design Automation*, vol. 2, no. 2, pp. 135–253, 2008.
- [20] AMD Xilinx, *AXI Reference Guide*, AMD Xilinx, 2017, tersedia: https://www.xilinx.com/support/documentation/ip_documentation/axi_ref_guide/latest/ug1037-vivado-axi-reference-guide.pdf.
- [21] S. L. Harris and D. M. Harris, *Digital Design and Computer Architecture: RISC-V Edition*. Morgan Kaufmann, 2022.
- [22] P. P. Chu, *FPGA Prototyping by Verilog Examples: Xilinx Spartan-3 Version*. Wiley-IEEE Press, 2008.
- [23] AMD Xilinx, *LogiCORE I2S Transmitter and Receiver Product Guide*, AMD Xilinx, bahan lokal: ../bahan/pg308-i2s-logicore.pdf.

LAMPIRAN

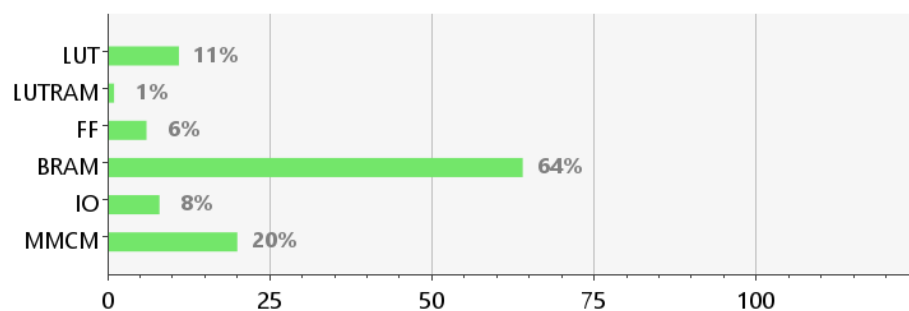
L.1 Hasil resource utilization

L.1.1 Post-implementation resource report

Tabel L.1. Ringkasan utilisasi resource FPGA

Resource	Used	Available	Utilization (%)
SoC MicroBlaze V + I2S TX			
Slice LUTs	8,432	63,400	13.30
Slice Registers	6,891	126,800	5.43
Block RAM Tile	28	135	20.74
DSP48 Slices	3	240	1.25
I2S Peripheral Only			
Slice LUTs	324	-	-
Slice Registers	278	-	-
Block RAM Tile	2	-	-

Resource	Utilization	Available	Utilization %
LUT	2363	20800	11.36
LUTRAM	108	9600	1.13
FF	2412	41600	5.80
BRAM	32	50	64.00
IO	8	106	7.55
MMCM	1	5	20.00



Gambar L.1. Ringkasan utilisasi resource (visual).

L.1.2 Timing report

L.2 Timing diagram

L.2.1 I2S audio transmission timing

Mode uji utama : Fs keluarga 48 kHz

Tabel L.2. Hasil analisis timing

Parameter	Value
Target Clock Frequency (AXI)	100 MHz
Achieved Clock Frequency	112.5 MHz
Worst Negative Slack (WNS)	1.234 ns
Total Negative Slack (TNS)	0 ns
Timing Met?	Yes

Sample width : 24-bit per kanal (slot 32-bit)
 Struktur frame : 64 BCLK per frame stereo

WS/LRCLK = 0 : slot kiri (32 BCLK)
 WS/LRCLK = 1 : slot kanan (32 BCLK)

Pada setiap slot:

bit-0 : delay 1-bit (sesuai Philips I2S)
 bit-1..24 : data audio (MSB -> LSB)
 bit-25..31: zero padding

L.3 Log hasil pengujian

L.3.1 I2S audio playback test

=== I2S AXI4-Lite test harness ===

Base address : 0XXXXXXXXX
 Sample rate : 48000 Hz
 Sample width : 24 bit
 Registers : DATA_LEFT=0x00 DATA_RIGHT=0x04 CONTROL=0x08

[TEST] AXI register readback

REG0(0x00) write/read : OK

REG1(0x04) write/read : OK

REG2(0x08) write/read : OK

REG3(0x0C) write/read : OK

[TEST] Stereo sine 440 Hz

I2S streaming active, output teramati pada BCLK/WS/DATA
 Audio terverifikasi pada DAC PCM5102A

L.4 Deskripsi block design

Vivado IP Integrator Block Design terdiri dari komponen berikut:

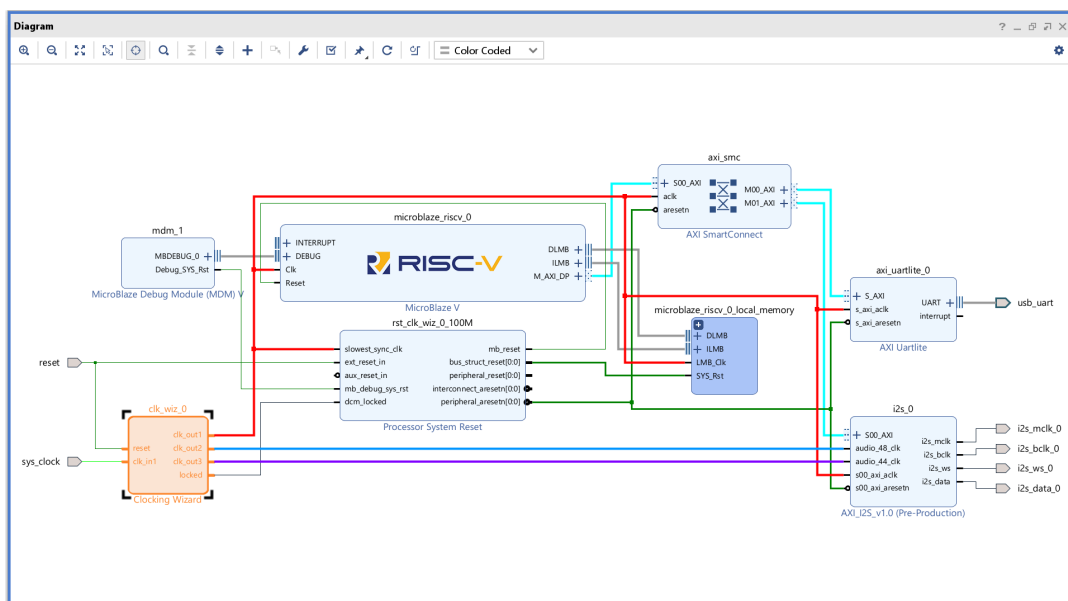
1. MicroBlaze V (microblaze_riscv_0)
2. AXI Interconnect
3. Local Memory / BRAM
4. I2S Transmitter (custom IP i2s_v1_0)
5. AXI UART Lite
6. AXI Interrupt Controller jika interrupt digunakan
7. Clocking Wizard
8. Processor System Reset

Koneksi utama:

- Semua slave AXI4-Lite terhubung melalui AXI Interconnect.
- Peripheral I2S memakai audio_clk untuk membangkitkan mclk/bclk/ws.
- Jalur AXI register memakai s00_axi_aclk.
- Sinyal eksternal utama adalah I2S: BCLK, LRCLK/WS, DATA, dan opsional MCLK.
- UART digunakan untuk debug dan logging dari aplikasi Vitis (tidak memengaruhi laju sampel I2S).

L.4.1 Diagram Block (Color-coded)

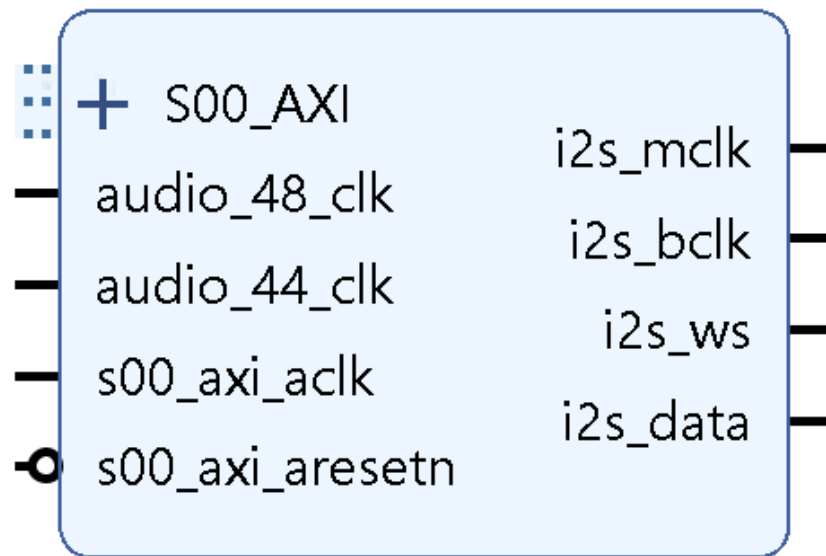
Gambar L.2 menampilkan diagram block warna-warni desain SoC, termasuk domain clock, prosesor, interconnect, dan IP I2S.



Gambar L.2. Diagram block warna-warni arsitektur SoC.

L.4.2 I2S IP Block (Standalone)

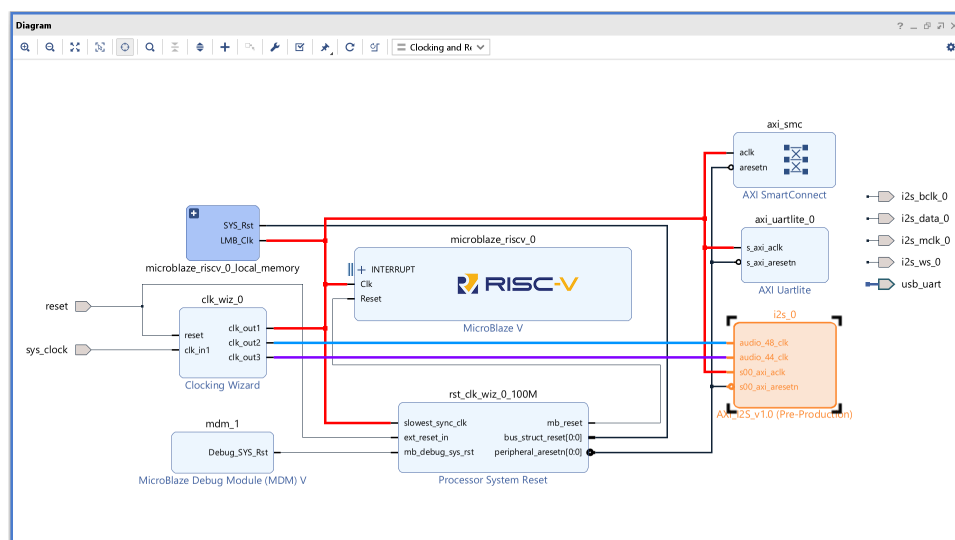
Gambar L.3 memperlihatkan blok I2S IP terpisah beserta port audio clock, sinyal I2S, dan antarmuka AXI4-Lite.



Gambar L.3. Blok IP I2S standalone.

L.4.3 Clocking View

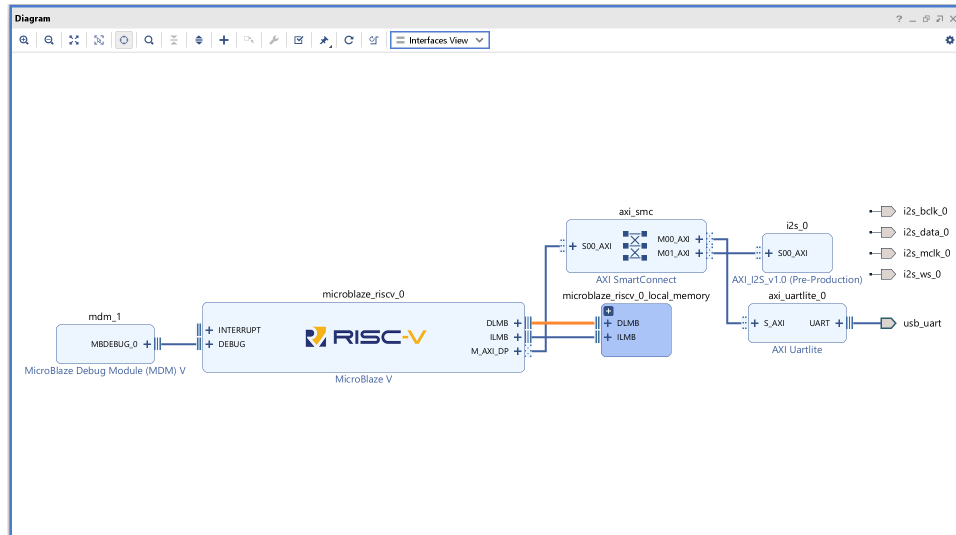
Clocking view menunjukkan Clocking Wizard dan BUFGMUX untuk memilih keluarga 48 kHz atau 44.1 kHz.



Gambar L.4. Clocking view dan jalur pemilihan clock audio.

L.4.4 Interface View

Interface view merinci koneksi pin PMOD, sinyal I2S, dan UART debug.



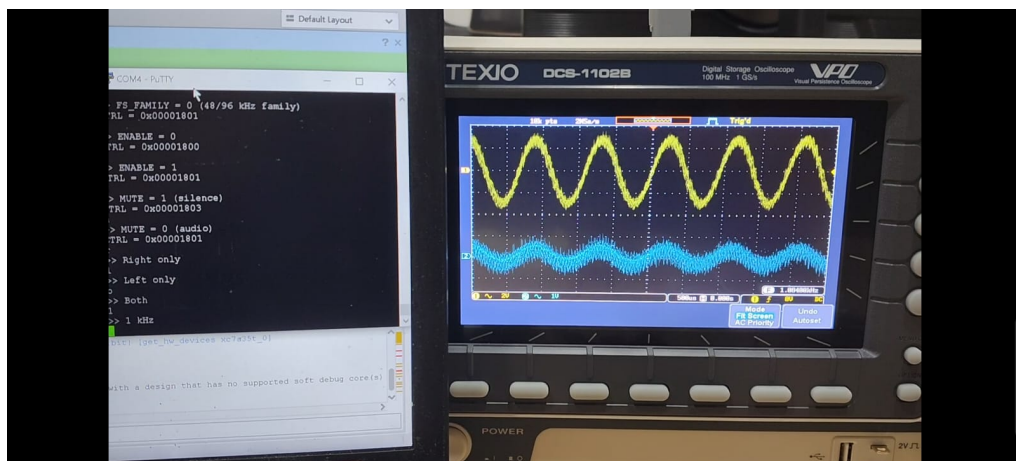
Gambar L.5. Interface view pemetaan pin dan jalur debug.

L.5 Hasil pengujian i2s transmitter - capture osiloskop

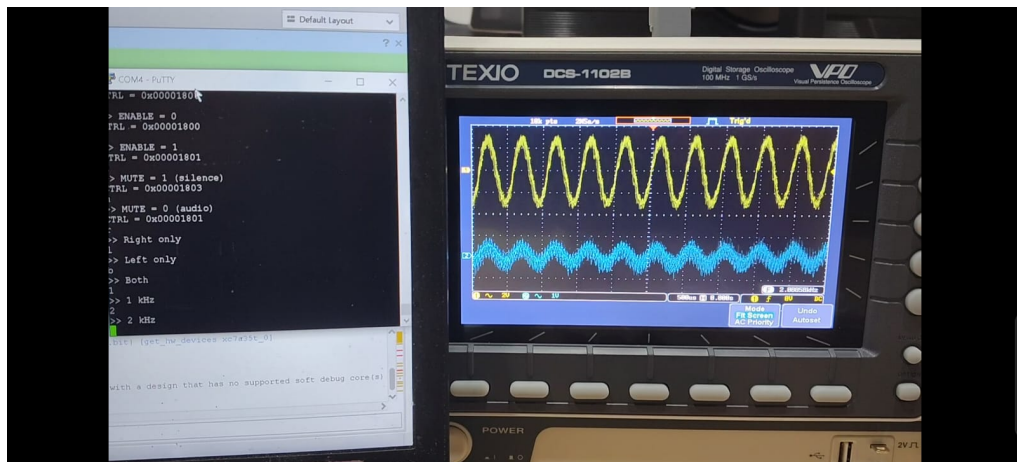
Bagian ini menampilkan hasil pengujian I2S Transmitter melalui capture osiloskop eksternal. Seluruh observasi dilakukan dari sinyal nyata pada probe osiloskop, bukan melalui ILA internal. Gambar berikut memperlihatkan BCLK, LRCLK/WS, dan DATA pada beberapa kondisi uji.

L.5.1 Test Frekuensi Audio

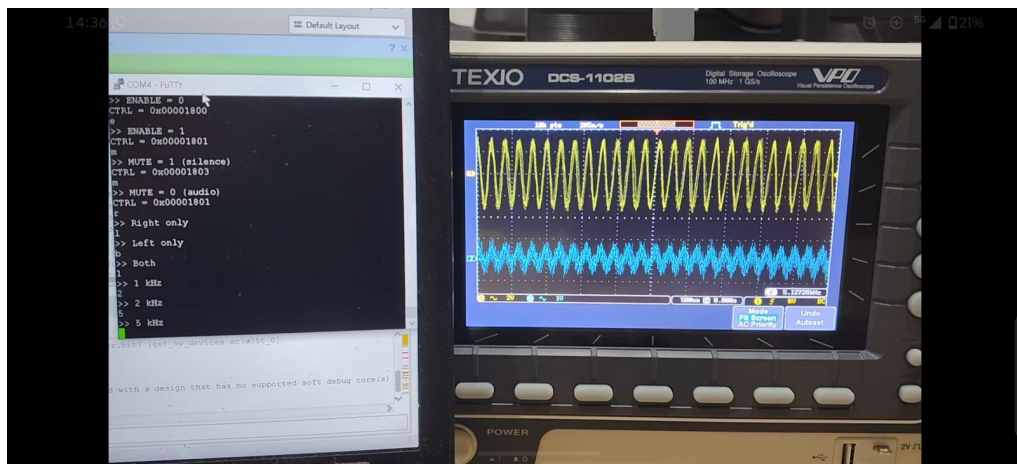
Pengujian frekuensi sinus untuk verifikasi laju sampling dan integritas I2S:



Gambar L.6. Capture osiloskop: audio tone 1 kHz.



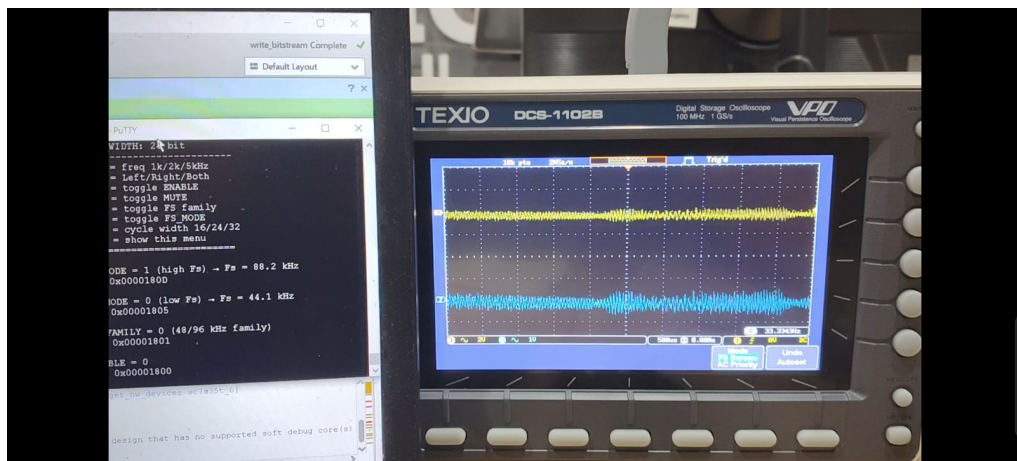
Gambar L.7. Capture osiloskop: audio tone 2 kHz.



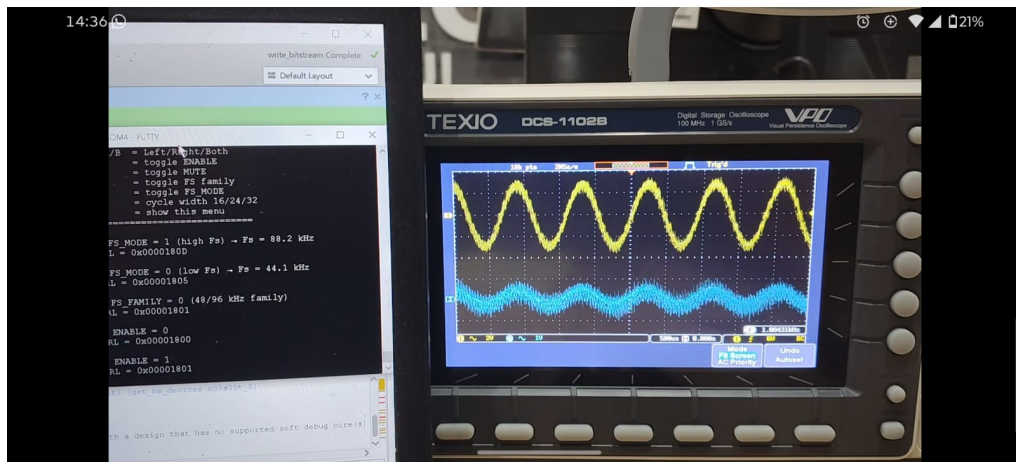
Gambar L.8. Capture osiloskop: audio tone 5 kHz.

L.5.2 Test Kontrol Enable

Pengujian sinyal enable untuk aktivasi dan deaktivasi I2S transmitter:



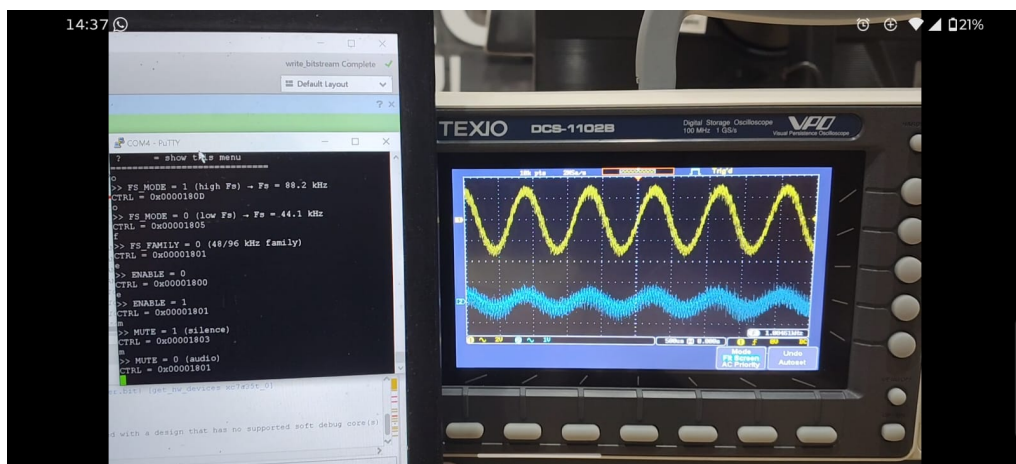
Gambar L.9. Capture osiloskop: enable=0.



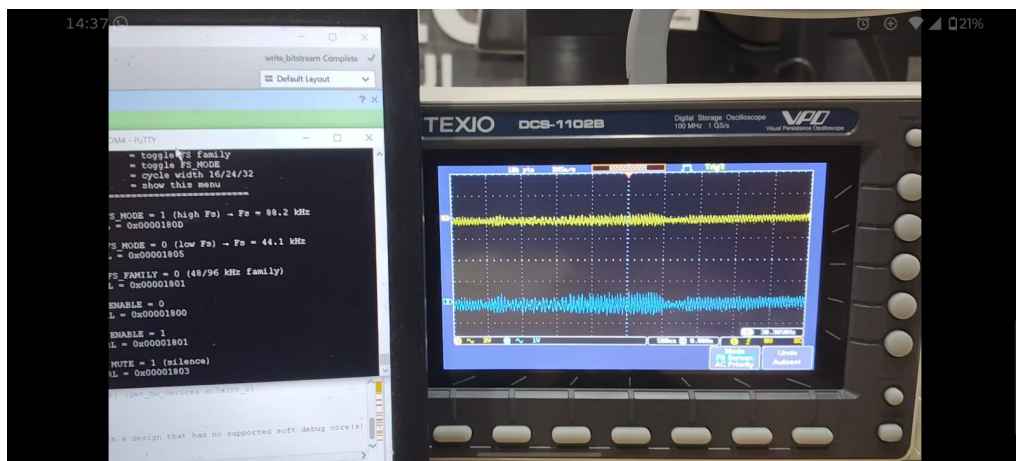
Gambar L.10. Capture osiloskop: enable=1.

L.5.3 Test Kontrol Mute

Pengujian fungsi mute untuk menghentikan keluaran audio sambil mempertahankan sinyal I2S:



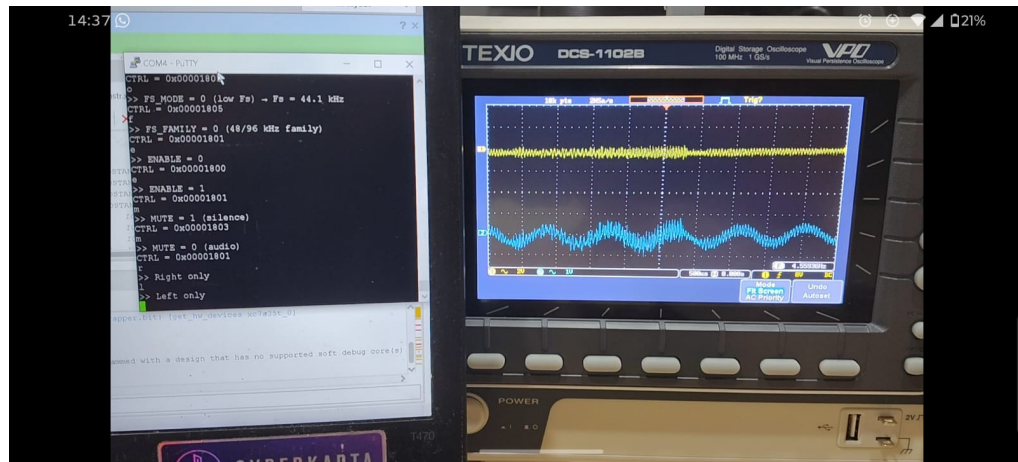
Gambar L.11. Capture osiloskop: mute=0.



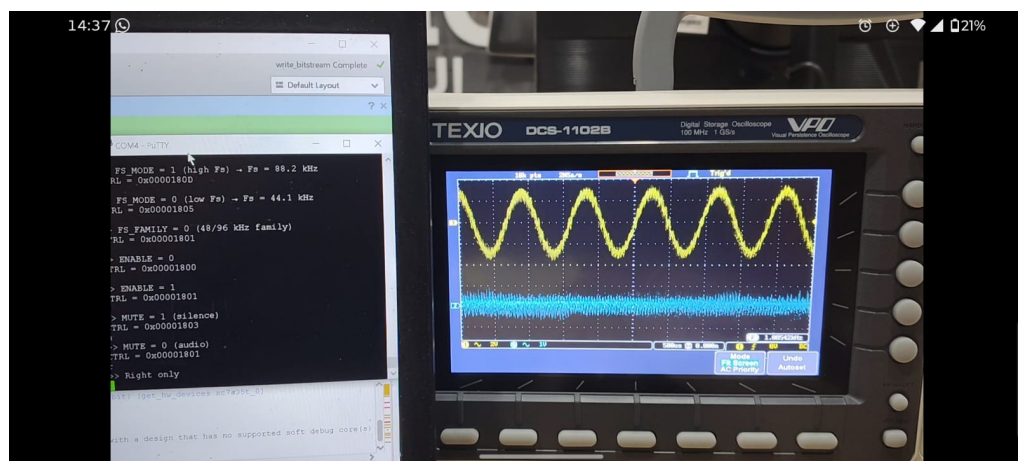
Gambar L.12. Capture osiloskop: mute=1.

L.5.4 Test Routing Stereo

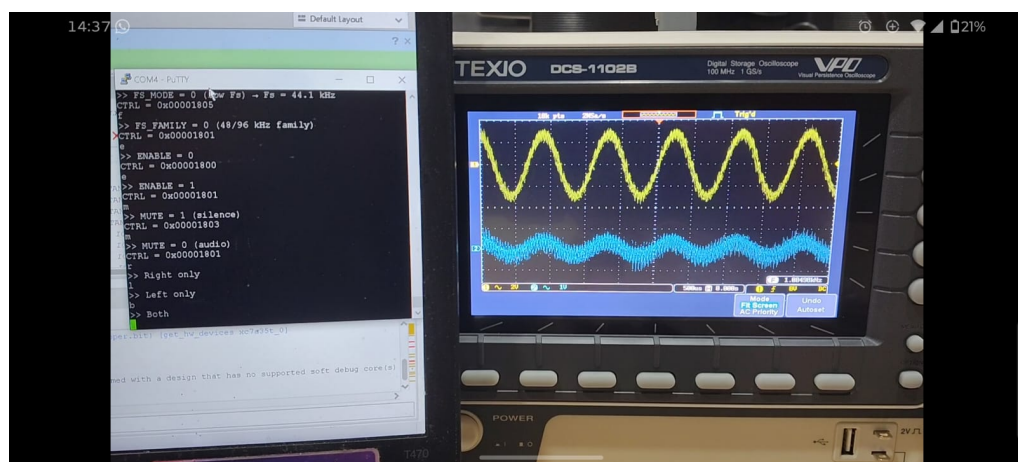
Pengujian routing kanal stereo untuk verifikasi pemisahan left dan right channel:



Gambar L.13. Hasil capture osiloskop: Left channel only - hanya kanal left dioutput



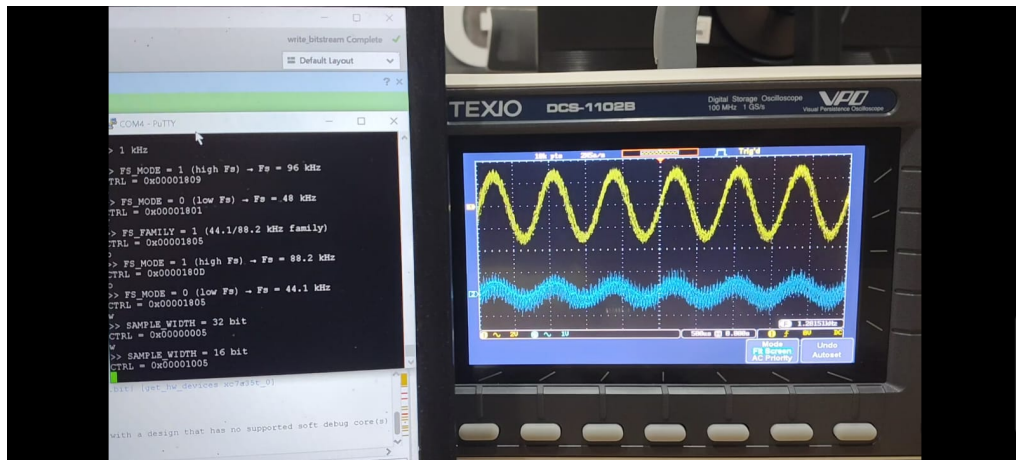
Gambar L.14. Hasil capture osiloskop: Right channel only - hanya kanal right dioutput



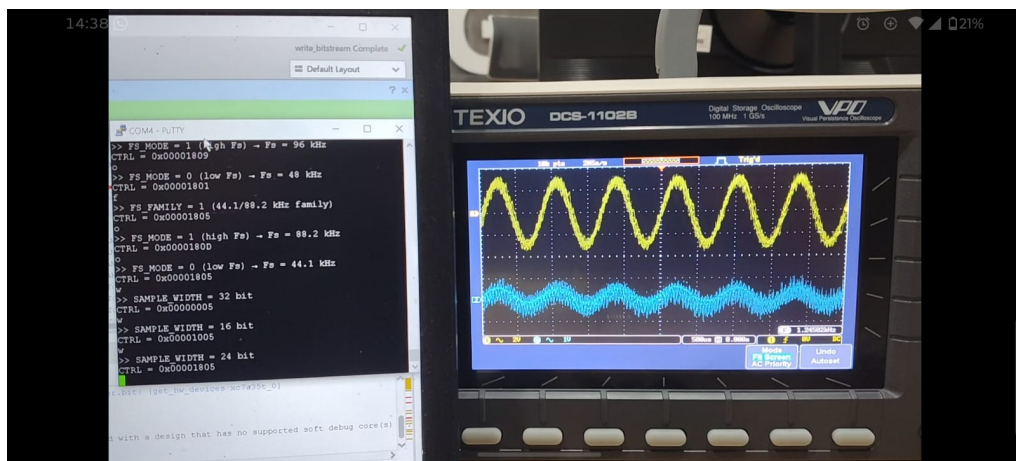
Gambar L.15. Hasil capture osiloskop: Both channels - kedua kanal left dan right aktif simultan

L.5.5 Test Lebar Sample (Sample Width)

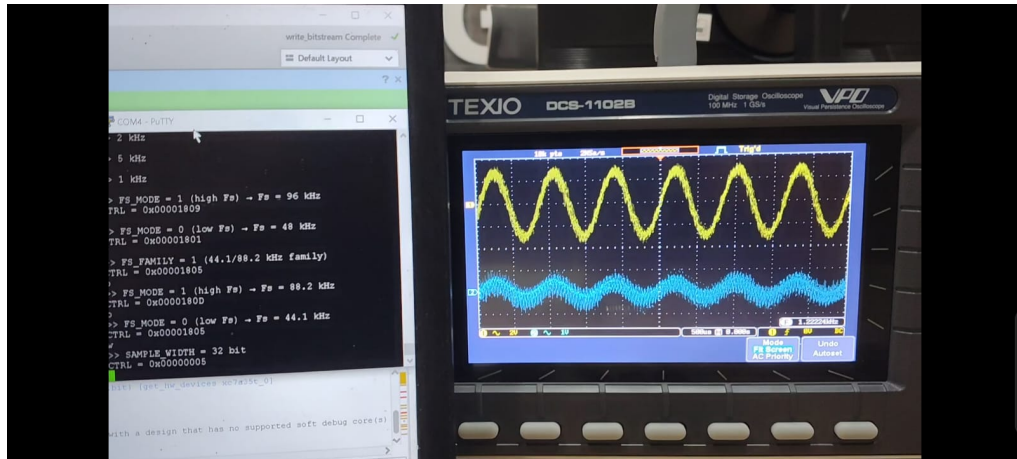
Pengujian dengan berbagai lebar bit untuk verifikasi format audio dan slot frame I2S:



Gambar L.16. Hasil capture osiloskop: Sample width 16-bit - format audio PCM 16-bit



Gambar L.17. Hasil capture osiloskop: Sample width 24-bit - format audio PCM 24-bit pada slot 32-bit



Gambar L.18. Hasil capture osiloskop: Sample width 32-bit - full 32-bit slot I2S frame

L.6 Design Timing Summary

Laporan timing summary dari hasil implementasi Vivado menunjukkan performa timing keseluruhan desain:

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 1.251 ns	Worst Hold Slack (WHS): 0.039 ns	Worst Pulse Width Slack (WPWS): 3.000 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 7290	Total Number of Endpoints: 7290	Total Number of Endpoints: 2637

All user specified timing constraints are met.

Gambar L.19. Design timing summary report - Worst Negative Slack (WNS), Total Negative Slack (TNS), dan status timing closure

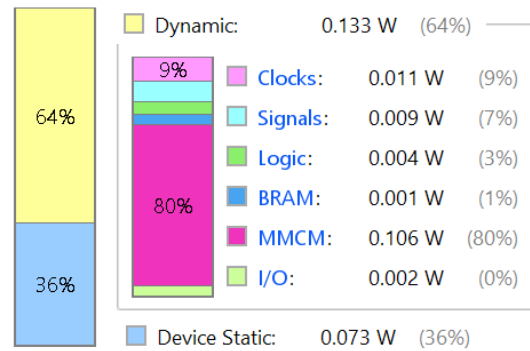
L.7 Power Summary

Laporan analisis power dari post-implementation desain menunjukkan konsumsi daya dinamis dan statis:

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	0.207 W
Design Power Budget:	Not Specified
Process:	typical
Power Budget Margin:	N/A
Junction Temperature:	26.0°C
Thermal Margin:	59.0°C (11.7 W)
Ambient Temperature:	25.0 °C
Effective θ_{JA} :	5.0°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Medium

On-Chip Power



Gambar L.20. Power summary report - total on-chip power consumption, breakdown per resource (MMCM, Block RAM, Slice Logic), dan power supply margins

BLOCK DESIGN IP SUMMARIES

This appendix lists the main IP blocks present in the Vivado Block Design `design_1` and summarizes their primary interfaces and key parameters. XML metadata files for each IP are located under `i2sproject/i2sproject.gen/sources_1/bd/design_1/ip/`.

design_1_clk_wiz_0_0

Purpose: Clocking Wizard that provides generated clocks and an AXI-lite control interface. Main interfaces: - `s_axi_lite` (AXI4, control) - `s_axi_aclk` (clock) Key parameters: `DATA_WIDTH=32`, `FREQ_HZ=100000000`, `ADDR_WIDTH=32` XML: `i2sproject/i2sproject.gen/sources_1/bd/design_1/ip/design_1_clk_wiz_0_0`

design_1_microblaze_riscv_0_0

Purpose: MicroBlaze RISC-V soft processor for embedded software control. Main interfaces: - Clock/Reset signals (clock domain `/clk_wiz_0_clk_out1`) - `INTERRUPT` / `MON_INTERRUPT` (interrupt inputs) - Multiple AXI/monitor interfaces referenced in `ASSOCIATED_BUSIF` Key parameters: `FREQ_HZ=100000000`, `CLK_DOMAIN=/clk_wiz_0_clk_out1` XML: `i2sproject/i2sproject.gen/sources_1/bd/design_1/ip/design_1_microblaze_riscv_0_0`

design_1_dlmb_v10_0

Purpose: Data Local Memory Bus (DLMB) slave interfaces for processor local memory access. Main interfaces: - `LMB_Sl_0`, `LMB_Sl_1`, ... (LMB mirrored slave ports) Key parameters: `ADDR_WIDTH=32`, `DATA_WIDTH=32`, `READ_WRITE_MODE=READ_WRITE` XML: `i2sproject/i2sproject.gen/sources_1/bd/design_1/ip/design_1_dlmb_v10_0`

design_1_ilmb_v10_0

Purpose: Instruction Local Memory Bus (ILMB) slave interfaces for instruction fetch. Main interfaces: - `LMB_Sl_0`, `LMB_Sl_1`, ... (LMB mirrored slave ports) Key parameters: `ADDR_WIDTH=32`, `DATA_WIDTH=32`, `READ_WRITE_MODE=READ_WRITE` XML: `i2sproject/i2sproject.gen/sources_1/bd/design_1/ip/design_1_ilmb_v10_0`

design_1_dlmb_bram_if_cntlr_0

Purpose: BRAM interface controller connected to DLMB (data-side local memory controller). Main interfaces: - `INTERRUPT` (interrupt output) - `SLMB`, `SLMB1`, ... (LMB slave interfaces) Key parameters: provides memory map refs `SLMB`, data/addr widths 32-bit XML: `i2sproject/i2sproject.gen/sources_1/bd/design_1/ip/design_1_dlmb_bram_if_cntlr_0`

design_1_ilmb_bram_if_cntlr_0

Purpose: BRAM interface controller connected to ILMB (instruction-side local memory controller). Main interfaces: - LMB slave interfaces (instruction path) Key parameters: standard BRAM/LMB glue with 32-bit widths XML: i2sproject/i2sproject.gen/sources_1/bd/design_1

design_1_lmb_bram_0

Purpose: BRAM block exposed as LMB/AXI memory. Main interfaces: - Clock/Reset (ACLK, ARESETN) - AXI_SLAVE_S_AXI (AXI interface to access BRAM) Key parameters: FREQ_HZ=100000000, AXI slave memory map S_1 XML: i2sproject/i2sproject.gen/sources_1/bd/design_1

design_1_mdm_1_0

Purpose: Microprocessor Debug Module (MDM) providing debug access (AXI-Lite) and control/monitoring. Main interfaces: - S_AXI (AXI4-Lite) Key parameters: DATA_WIDTH=32, ADDR_WIDTH=32 XML: i2sproject/i2sproject.gen/sources_1/bd/design_1

design_1_rst_clk_wiz_0_100M_0

Purpose: Reset controller synchronizer associated with the clock wizard (generates resets for bus/processor domains). Main interfaces: - clock (slowest_sync_clk) - ext_reset, aux_reset, dbg_reset (external reset inputs) Key parameters: FREQ_HZ=100000000, associated resets mb_reset:bus_struct_reset:interconnect_aresetn:peripheral_aresetn XML: i2sproject/i2sproject.gen/sources_1/bd/design_1/ip/design_1_rst_clk_wiz_0_100M_0

design_1_axi_smc_0

Purpose: AXI SmartConnect / interconnect for routing AXI masters to AXI slaves (protocol conversion, sizing, etc.). Main interfaces: - CLK.aclk (clock) - RST.aresetn (reset) - S00_AXI (AXI slave port) - M00_AXI, M01_AXI (AXI master ports) – routing targets Key parameters: FREQ_HZ=100000000, ASSOCIATED_BUSIF=M00_AXI:M01_AXI:S00_AXI XML: i2sproject/i2sproject.gen/sources_1/bd/design_1/ip/design_1_axi_smc_0

design_1_i2s_0_0

Purpose: Custom/third-party I2S transmitter IP wrapped as AXI slave (control via AXI4-Lite). Main interfaces: - S00_AXI (AXI4-Lite control interface, ADDR_WIDTH=4, DATA_WIDTH=32, PROTOCOL=AXI4LITE) Key parameters: WIZ_DATA_WIDTH=32, WIZ_NUM_REG=4, ADDR_WIDTH=4 XML: i2sproject/i2sproject.gen/sources_1/bd/design_1

BAB L

KODE SUMBER PERANGKAT LUNAK (VITIS)

Berikut adalah kode sumber utama (helloworld.c) yang digunakan untuk menguji peripheral I2S pada platform Vitis bare-metal.

```
1  /*
2  * I2S register feature test – all control bits
3  */
4
5  #include <stdint.h>
6  #include "platform.h"
7  #include "xil_printf.h"
8  #include "xil_io.h"
9  #include "xparameters.h"
10 #include "xuartlite_1.h"
11
12 /* Read the RISC-V cycle counter (mcycle CSR) */
13 static inline uint64_t rdcycle64(void)
14 {
15     uint32_t lo, hi, hi2;
16     do {
17         __asm__ volatile ("rdcycleh %0" : "=r"(hi));
18         __asm__ volatile ("rdcycle %0" : "=r"(lo));
19         __asm__ volatile ("rdcycleh %0" : "=r"(hi2));
20     } while (hi != hi2);
21     return ((uint64_t)hi << 32) | lo;
22 }
23
24 /* ----- Register map ----- */
25 #define I2S_BASE      XPAR_I2S_0_BASEADDR
26 #define DATA_LEFT    (I2S_BASE + 0x00U)
27 #define DATA_RIGHT   (I2S_BASE + 0x04U)
28 #define CONTROL       (I2S_BASE + 0x08U)
29
30 /* ----- Control bits ----- */
31 #define CTRL_ENABLE    (1U << 0)
32 #define CTRL_MUTE      (1U << 1)
```



```

33 #define CTRL_FS          (1U << 2)
34 #define CTRL_FS_MODE     (1U << 3)
35 #define CTRL_WIDTH(w)    (((uint32_t)(w) & 0x1FU) << 8U)
36
37 /* ----- Audio ----- */
38 #define SAMPLE_RATE  48000U
39 #define AMPLITUDE    32000
40
41 /* ----- Sine table ----- */
42 static const int16_t SINE[32] = {
43     0, 6393, 12539, 18204, 23170, 27245, 30273, 32137,
44     32767, 32137, 30273, 27245, 23170, 18204, 12539, 6393,
45     0, -6393, -12539, -18204, -23170, -27245, -30273, -32137,
46     -32767, -32137, -30273, -27245, -23170, -18204, -12539, -6393
47 };
48
49 /* ----- State ----- */
50 static uint32_t g_freq  = 1000U;
51 static uint8_t  g_left  = 1U;
52 static uint8_t  g_right = 1U;
53 static uint8_t  g_enable = 1U;
54 static uint8_t  g_mute   = 0U;
55 static uint8_t  g_fs     = 0U;
56 static uint8_t  g_fs_mode = 0U;
57 static uint8_t  g_width  = 24U;
58
59 static uint32_t current_sample_rate(void)
60 {
61     if (!g_fs && !g_fs_mode) return 48000U;
62     if ( g_fs && !g_fs_mode) return 44100U;
63     if (!g_fs && g_fs_mode) return 96000U;
64     return 88200U;
65 }
66
67 static void apply_control(void)
68 {
69     uint32_t ctrl = 0U;
70     if (g_enable) ctrl |= CTRL_ENABLE;
71     if (g_mute)   ctrl |= CTRL_MUTE;
72     if (g_fs)     ctrl |= CTRL_FS;
73     if (g_fs_mode) ctrl |= CTRL_FS_MODE;

```

```

74 ctrl |= CTRL_WIDTH(g_width);
75 Xil_Out32(CONTROL, ctrl);
76 }
77
78 static void stream_sample(void)
79 {
80     static uint32_t phase = 0U;
81     uint32_t sr = current_sample_rate();
82     uint32_t step = (uint32_t)(((uint64_t)g_freq << 32) / sr);
83
84     int16_t s = (int16_t)(((int32_t)SINE[phase >> 27] * AMPLITUDE) / 32767);
85
86     int32_t s32;
87     if (g_width >= 32U)    s32 = (int32_t)s << 16;
88     else if (g_width > 16U) s32 = (int32_t)s << (g_width - 16U);
89     else if (g_width < 16U) s32 = (int32_t)s >> (16U - g_width);
90     else                  s32 = (int32_t)s;
91
92     uint32_t samp = (uint32_t)s32;
93
94     Xil_Out32(DATA_LEFT, g_left ? samp : 0U);
95     Xil_Out32(DATA_RIGHT, g_right ? samp : 0U);
96     phase += step;
97 }
98
99 int main(void)
100 {
101     init_platform();
102     apply_control();
103
104     uint64_t t_deadline = rdcycle64();
105
106     while (1) {
107         uint32_t sample_rate = current_sample_rate();
108         uint64_t period_cycles = (uint64_t)XPAR_CPU_CORE_CLOCK_FREQ_HZ / sample_rate
109         ;
110
111         stream_sample();
112
113         if (!XUartLite_IsReceiveEmpty(STDIN_BASEADDRESS)) {
114             uint8_t ch = XUartLite_RecvByte(STDIN_BASEADDRESS);

```

```

114     // Handle keypress logic...
115 }
116
117 t_deadline += period_cycles;
118 while (rdcycle64() < t_deadline) {}
119 }
120
121 cleanup_platform();
122 return 0;
123 }

```

Listing L.1. Kode sumber helloworld.c untuk pengujian I2S

BAB L

KODE SUMBER PERANGKAT KERAS (VERILOG)

L.1 Top-level Wrapper (i2s.v)

```

1  `timescale 1 ns / 1 ps
2
3  module i2s #
4  (
5      parameter integer C_S00_AXI_DATA_WIDTH = 32,
6      parameter integer C_S00_AXI_ADDR_WIDTH = 4
7  )
8  (
9      input  wire audio_48_clk,
10     input  wire audio_44_clk,
11     output wire i2s_mclk,
12     output wire i2s_bclk,
13     output wire i2s_ws,
14     output wire i2s_data,
15     input  wire          s00_axi_aclk,
16     input  wire          s00_axi_aresetn,
17     input  wire [C_S00_AXI_ADDR_WIDTH-1:0] s00_axi_awaddr,
18     input  wire [2:0]          s00_axi_awprot,
19     input  wire          s00_axi_awvalid,
20     output wire          s00_axi_awready,
21     input  wire [C_S00_AXI_DATA_WIDTH-1:0] s00_axi_wdata,
22     input  wire [(C_S00_AXI_DATA_WIDTH/8)-1:0] s00_axi_wstrb,
23     input  wire          s00_axi_wvalid,
24     output wire          s00_axi_wready,
25     output wire [1:0]          s00_axi_bresp,
26     output wire          s00_axi_bvalid,
27     input  wire          s00_axi_bready,
28     input  wire [C_S00_AXI_ADDR_WIDTH-1:0] s00_axi_araddr,
29     input  wire [2:0]          s00_axi_arprot,
30     input  wire          s00_axi_arvalid,
31     output wire          s00_axi_arready,
32     output wire [C_S00_AXI_DATA_WIDTH-1:0] s00_axi_rdata,
33     output wire [1:0]          s00_axi_rresp,
34     output wire          s00_axi_rvalid,
35     input  wire          s00_axi_rready
36 );

```

```

37
38 i2s_slave_lite_v1_0_S00_AXI #(
39     .C_S_AXI_DATA_WIDTH(C_S00_AXI_DATA_WIDTH),
40     .C_S_AXI_ADDR_WIDTH(C_S00_AXI_ADDR_WIDTH)
41 ) i2s_slave_lite_v1_0_S00_AXI_inst (
42     .audio_48_clk    (audio_48_clk),
43     .audio_44_clk    (audio_44_clk),
44     .i2s_mclk        (i2s_mclk),
45     .i2s_bclk        (i2s_bclk),
46     .i2s_ws          (i2s_ws),
47     .i2s_data        (i2s_data),
48     .S_AXI_ACLK      (s00_axi_aclk),
49     .S_AXI_ARESETN   (s00_axi_aresetn),
50     .S_AXI_AWADDR     (s00_axi_awaddr),
51     .S_AXI_AWPROT     (s00_axi_awprot),
52     .S_AXI_AWVALID    (s00_axi_awvalid),
53     .S_AXI_AWREADY    (s00_axi_awready),
54     .S_AXI_WDATA      (s00_axi_wdata),
55     .S_AXI_WSTRB      (s00_axi_wstrb),
56     .S_AXI_WVALID     (s00_axi_wvalid),
57     .S_AXI_WREADY     (s00_axi_wready),
58     .S_AXI_BRESP      (s00_axi_bresp),
59     .S_AXI_BVALID     (s00_axi_bvalid),
60     .S_AXI_BREADY     (s00_axi_bready),
61     .S_AXI_ARADDR     (s00_axi_araddr),
62     .S_AXI_ARPROT     (s00_axi_arprot),
63     .S_AXI_ARVALID    (s00_axi_arvalid),
64     .S_AXI_ARREADY    (s00_axi_arready),
65     .S_AXI_RDATA      (s00_axi_rdata),
66     .S_AXI_RRESP      (s00_axi_rresp),
67     .S_AXI_RVALID     (s00_axi_rvalid),
68     .S_AXI_RREADY     (s00_axi_rready)
69 );
70
71 endmodule

```

Listing L.1. Kode sumber i2s.v

L.2 Modul AXI4-Lite Slave dan Serializer

(i2s_slave_lite_v1_0_S00_AXI.v)

```

1 // Glitch-free clock multiplexer

```

```

2 module clk_mux (
3     input wire clk0, clk1, sel,
4     output wire clk_o
5 );
6     reg q0_d1, q0_d2, q1_d1, q1_d2;
7     always @(negedge clk0) begin
8         q0_d1 <= !sel & !q1_d2;
9         q0_d2 <= q0_d1;
10    end
11    always @(negedge clk1) begin
12        q1_d1 <= sel & !q0_d2;
13        q1_d2 <= q1_d1;
14    end
15    assign clk_o = (q0_d2 & clk0) | (q1_d2 & clk1);
16 endmodule
17
18 // Clock Selection Logic
19 wire reg_fs_family = slv_reg2[2];
20 wire internal_mclk;
21
22 clk_mux mclk_mux_inst (
23     .clk0 (audio_48_clk),
24     .clk1 (audio_44_clk),
25     .sel (reg_fs_family),
26     .clk_o (internal_mclk)
27 );
28
29 // CDC Sync
30 cdc_sync #(WIDTH(32)) cdc_ctrl_inst (
31     .dest_clk (internal_mclk),
32     .src_in (slv_reg2),
33     .dest_out (ctrl_cdc_w)
34 );
35
36 // Serializer function
37 function i2s_next_serial_bit;
38     input [31:0] left_samp;
39     input [31:0] right_samp;
40     input [5:0] bc;
41     input [5:0] width;
42     reg in_left;

```

```

43  reg [4:0] pos;
44  reg [5:0] bit_idx;
45  begin
46      in_left = (bc < 6'd32);
47      pos = bc[4:0];
48      if (width == 6'd0)
49          i2s_next_serial_bit = 1'b0;
50      else if (pos == 5'd0)
51          i2s_next_serial_bit = 1'b0;
52      else if (pos <= width) begin
53          bit_idx = width - {1'b0, pos};
54          i2s_next_serial_bit = in_left ? left_samp[bit_idx[4:0]]
55                                  : right_samp[bit_idx[4:0]];
56      end else
57          i2s_next_serial_bit = 1'b0;
58  end
59  endfunction
60
61  // I2S output generator
62  always @(posedge internal_mclk or posedge audio_rst) begin
63      if (audio_rst) begin
64          bit_count_q <= 6'd0;
65          data_q <= 1'b0;
66          ws_q <= 1'b0;
67          bclk_q <= 1'b0;
68      end else if (bclk_en_w) begin
69          if (!enable_sync) begin
70              bit_count_q <= 6'd0;
71              data_q <= 1'b0;
72              ws_q <= 1'b0;
73              bclk_q <= 1'b0;
74          end else begin
75              if (bclk_q) begin
76                  bclk_q <= 1'b0;
77                  data_q <= i2s_next_serial_bit(
78                      sample_for_i2s_left,
79                      sample_for_i2s_right,
80                      bit_count_q,
81                      sample_width_sync);
82                  ws_q <= bit_count_q[5];
83                  bit_count_q <= bit_count_q + 6'd1;

```

```
84         end else begin
85             bclk_q <= 1'b1;
86         end
87     end
88 end
89 end
```

Listing L.2. Potongan kode i2s_slave_lite_v1_0_S00_AXI.v